

g·NEED ACCESS

NETWORK ENABLED EASY
DATA ACCESS



INSTRUCTIONS FOR USE



CLIENT API



User Manual: g.NEEDaccess Client API

Version 1.24.00 | Revision 1.0 | 2024

Copyright © g.tec medical engineering GmbH

www.gtec.at | office@gtec.at

CONTENTS

1 THE CLIENT API	8
1.1 TERMS AND DISAMBIGUATION.....	8
1.1.1 DEVICE	9
1.1.2 MASTER/SLAVE	9
1.1.3 SERVER/CLIENT	9
1.1.4 DATA ACQUISITION UNIT (DAQ UNIT)	10
1.1.5 DATA ACQUISITION SESSION (DAQ SESSION)	10
1.1.6 SESSION CREATOR/PARTICIPANT	10
1.1.7 CONNECTION HANDLE	11
1.1.8 GDS	11
1.2 USING THE C CLIENT API LIBRARIES.....	11
1.2.1 INCLUDE HEADER FILES	11
1.2.2 LINK AGAINST ENTRY POINT INFORMATION (LIB)	12
1.2.3 EXECUTE WITH DYNAMIC LINK LIBRARIES (DLLS)	12
1.3 GENERAL DATA ACQUISITION PROCEDURE.....	13
1.4 THE GDSCLIENTAPIDEMO PROJECT.....	13
1.4.1 COMPILING THE GDSCLIENTAPIDEMO PROJECT.....	13
1.4.2 READING THE RECORDED DATA FILE.....	14
1.5 THE GDSMULTICLIENTDEMO PROJECT.....	14
1.5.1 COMPILING THE GDSMULTICLIENTDEMO PROJECT.....	14
1.5.2 READING THE RECORDED DATA FILE.....	15
2 CLIENT API – C REFERENCE	16
2.1 COMMON FUNCTIONALITY (GDSCLIENTAPI.H)	16
2.1.1 CONSTANTS.....	16
ERROR_MESSAGE_LENGTH_MAX.....	16
DEVICE_NAME_LENGTH_MAX.....	16
IP_ADDRESS_LENGTH_MAX	16
2.1.2 ERROR CODES	16
GDS_ERROR_SUCCESS.....	16
GDS_ERROR_INVALID_PARAMETER.....	16
GDS_ERROR_INVALID_HANDLE.....	16
GDS_ERROR_HANDLE_NOT_ASSOCIATED	16
GDS_ERROR_HANDLE_ALREADY_ASSOCIATED.....	16
GDS_ERROR_ACCESS_DENIED	16
GDS_ERROR_INVALID_DEVICE_LIST	16
GDS_ERROR_INVALID_DEVICE.....	17
GDS_ERROR_OPEN_DEVICE	17
GDS_ERROR_DEVICE_ALREADY_IN_USE.....	17
GDS_ERROR_INVALID_CONFIGURATION.....	17
GDS_ERROR_INCOMPATIBLE_CONFIGURATIONS.....	17

GDS_ERROR_READ_CONFIGURATION.....	17
GDS_ERROR_INVALID_ACQUISITION_STATE.....	17
GDS_ERROR_INVALID_STREAMING_STATE	17
GDS_ERROR_BUFFER_TOO_SMALL.....	17
GDS_ERROR_ACQUISITION_BUFFER_OVERFLOW.....	17
GDS_ERROR_TRANSMISSION_BUFFER_OVERFLOW	17
GDS_ERROR_ACQUISITION_DIED	17
GDS_ERROR_SERIALIZATION.....	17
GDS_ERROR_INVALID_ENDPOINT	18
GDS_ERROR_INVALID_REQUEST_STATUS	18
GDS_ERROR_REQUEST_TIMED_OUT	18
GDS_ERROR_UNKNOWN.....	18
2.1.3 TYPE DEFINITIONS	18
NULL	18
BOOL	18
FALSE.....	18
TRUE	18
GDS_HANDLE	18
2.1.4 ENUMERATIONS.....	18
GDS_DEVICE_TYPE.....	18
GDS_FILTER_TYPE	18
GDS_CHANNEL_DIRECTION.....	19
GDS_ELECTRODE_TYPE.....	19
2.1.5 FUNCTION POINTERS.....	20
GDS_Callback.....	20
GDS_ResultCallback.....	20
2.1.6 STRUCTURES.....	21
GDS_RESULT.....	21
GDS_ENDPOINT.....	21
GDS_DEVICE_INFO	21
GDS_DEVICE_CONNECTION_INFO	21
GDS_CONFIGURATION_BASE.....	21
GDS_FILTER_INFO.....	22
2.1.7 FUNCTIONS.....	23
GDS_Initialize	23
GDS_Uninitialize.....	24
GDS_GetConnectedDevices	25
GDS_FreeConnectedDevicesList	26
GDS_Connect	27
GDS_Disconnect.....	30
GDS_BecomeCreator	31
GDS_BecomeParticipant.....	32
GDS_SetConfiguration	33
GDS_GetConfiguration	35

GDS_FreeConfigurationList	36
GDS_StartAcquisition	37
GDS_StopAcquisition	38
GDS_StartStreaming	39
GDS_StopStreaming	40
GDS_GetDataInfo	41
GDS_GetData	43
2.1.8 CALLBACK REGISTRATION FUNCTIONS	45
GDS_SetConfigurationChangedCallback	45
GDS_SetDataAcquisitionStartedCallback	46
GDS_SetDataAcquisitionStoppedCallback	47
GDS_SetDataAcquisitionErrorCallback	48
GDS_SetDataReadyCallback	49
GDS_SetSessionCreatorDiedCallback	50
GDS_SetNewSessionCreatorCallback	51
GDS_SetServerDiedCallback	52
GDS_SetForcedClientShutdownCallback	53
2.2 G.USBAMP SPECIFICS (GDSCLIENTAPI_GUSBAMP.H)	54
2.2.1 CONSTANTS	54
GDS_GUSBAMP_CHANNELS_MAX	54
GDS_GUSBAMP_GROUPS_MAX	54
GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNELS_MAX	54
GDS_GUSBAMP_DEVICE_INFORMATION_LENGTH_MAX	54
2.2.2 ENUMERATIONS	55
GDS_GUSBAMP_WAVESHAPE	55
2.2.3 STRUCTURES	55
GDS_GUSBAMP_SAMPLING_RATE_FEATURES	55
GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNEL	55
GDS_GUSBAMP_SCALING	55
GDS_GUSBAMP_SIGNAL_GENERATOR_CONFIGURATION	55
GDS_GUSBAMP_CHANNEL_CONFIGURATION	56
GDS_GUSBAMP_CONFIGURATION	56
2.2.4 FUNCTIONS	59
GDS_GUSBAMP_GetDeviceInformation	59
GDS_GUSBAMP_GetSupportedSamplingRates	60
GDS_GUSBAMP_GetBandpassFilters	62
GDS_GUSBAMP_GetNotchFilters	64
GDS_GUSBAMP_GetAsyncDigitalIOs	66
GDS_GUSBAMP_SetAsyncDigitalOutputs	67
GDS_GUSBAMP_GetScaling	69
GDS_GUSBAMP_SetScaling	70
GDS_GUSBAMP_Calibrate	71
GDS_GUSBAMP_GetImpedance	72

GDS_GUSBAMP_GetImpedanceEx	74
2.3 G.HIAMP SPECIFICS (GDSCLIENTAPI_GHIAMP.H)	76
2.3.1 CONSTANTS.....	76
GDS_GHIAMP_CHANNELS_MAX	76
GDS_GHIAMP_DEVICE_INFORMATION_LENGTH_MAX	76
2.3.2 STRUCTURES.....	76
GDS_GHIAMP_SAMPLING_RATE_FEATURES.....	76
GDS_GHIAMP_SCALING	76
GDS_GHIAMP_SIGNAL_GENERATOR_CONFIGURATION.....	76
GDS_GHIAMP_CHANNEL_CONFIGURATION.....	76
GDS_GHIAMP_CONFIGURATION	77
2.3.3 FUNCTIONS.....	79
GDS_GHIAMP_GetDeviceInformation.....	79
GDS_GHIAMP_GetAvailableChannels	80
GDS_GHIAMP_GetSupportedSamplingRates	81
GDS_GHIAMP_GetBandpassFilters.....	83
GDS_GHIAMP_GetNotchFilters	85
GDS_GHIAMP_GetFactoryScaling.....	87
GDS_GHIAMP_GetScaling.....	88
GDS_GHIAMP_SetScaling.....	89
GDS_GHIAMP_Calibrate	90
GDS_GHIAMP_GetImpedance	91
GDS_GHIAMP_GetImpedanceEx.....	94
2.4 G.NAUTILUS SPECIFICS (GDSCLIENTAPI_GNAUTILUS.H).....	97
2.4.1 CONSTANTS.....	97
GDS_GNAUTILUS_MODULES_MAX.....	97
GDS_GNAUTILUS_CHANNELS_PER_MODULE_MAX.....	97
GDS_GNAUTILUS_CHANNELS_MAX.....	97
GDS_GNAUTILUS_ADDITIONAL_CHANNELS_MAX	97
GDS_GNAUTILUS_DIGITAL_IO_CHANNELS_MAX.....	97
GDS_GNAUTILUS_ELECTRODE_NAME_LENGTH_MAX	97
GDS_GNAUTILUS_DEVICE_INFORMATION_LENGTH_MAX.....	97
2.4.2 ENUMERATIONS.....	98
GDS_GNAUTILUS_INPUT_SIGNAL	98
2.4.3 FUNCTION POINTERS	98
GDS_GNAUTILUS_ImpedanceMeasurementCallback	98
2.4.4 STRUCTURES.....	99
GDS_GNAUTILUS_DIGITAL_IO_CHANNEL	99
GDS_GNAUTILUS_SCALING.....	99
GDS_GNAUTILUS_CHANNEL_CONFIGURATION.....	99
GDS_GNAUTILUS_CONFIGURATION.....	99
2.4.5 FUNCTIONS.....	102
GDS_GNAUTILUS_GetDeviceInformation.....	102

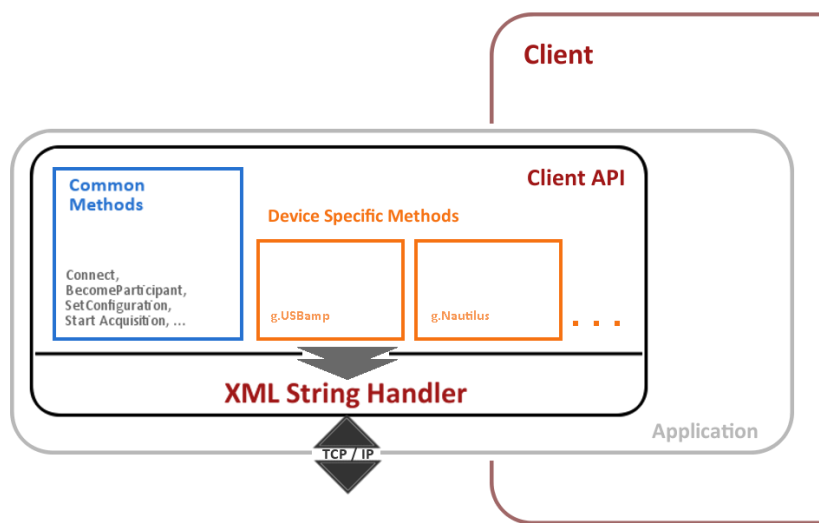
GDS_GNAUTILUS_GetChannelNames.....	103
GDS_GNAUTILUS_GetAvailableChannels.....	105
GDS_GNAUTILUS_GetAvailableDigitalIOs	106
GDS_GNAUTILUS_GetSupportedSamplingRates.....	107
GDS_GNAUTILUS_GetSupportedSensitivities.....	109
GDS_GNAUTILUS_GetSupportedNetworkChannels.....	110
GDS_GNAUTILUS_GetSupportedInputSources.....	111
GDS_GNAUTILUS_SetNetworkChannel	112
GDS_GNAUTILUS_GetNetworkChannel.....	113
GDS_GNAUTILUS_GetScaling	114
GDS_GNAUTILUS_SetScaling.....	115
GDS_GNAUTILUS_ResetScaling	116
GDS_GNAUTILUS_Calibrate.....	117
GDS_GNAUTILUS_GetBandpassFilters	118
GDS_GNAUTILUS_GetNotchFilters.....	120
GDS_GNAUTILUS_GetImpedance.....	122
GDS_GNAUTILUS_GetImpedanceEx	124
GDS_GNAUTILUS_StartImpedanceMeasurement	127
GDS_GNAUTILUS_StartImpedanceMeasurementEx.....	129
GDS_GNAUTILUS_StopImpedanceMeasurement	131

1 THE CLIENT API

The Client API is a library that provides all necessary methods for data acquisition using the server service. It consists of common methods that can be applied to any kind of DAQ unit and additional device specific methods for special operations.

The Client API can be used by any application that represents a client.

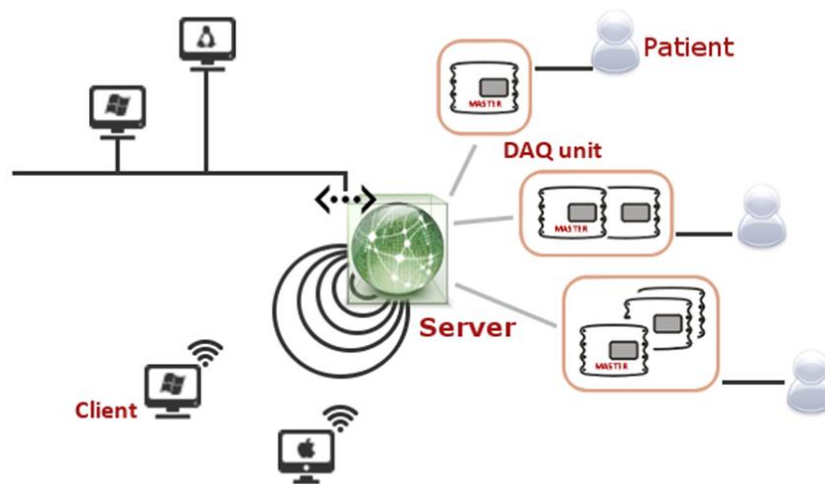
The API communicates with the server service using XML strings wrapped into TCP/IP messages. The strings carry information like commands, configurations or acquisition data. The unique connection handle is always part of the XML string to identify the client.



1.1 TERMS AND DISAMBIGUATION

Several terms in the g.NEEDaccess documentation are introduced to distinguish between several components and logical concepts of the server. This chapter lists some recurring terms that are used consistently throughout all g.NEEDaccess documentation.

The following figure illustrates the working principle of the server. Clients can connect to the server either via a wired or a wireless network. In case a client runs on the same computer as the server service, a loopback is used (IP address: 127.0.0.1) to address the machine's own Ethernet adapter.



1.1.1 DEVICE

A device (or DAQ device) is a physical device, namely a bio signal amplifier, that measures data and sends it to the computer it is connected to. It can be of type g.USBamp, g.HIamp or g.Nautilus.

1.1.2 MASTER/SLAVE

When multiple devices, usually of the same type, are hardware-synchronized (i.e. one of these devices synchronizes data acquisition among all other devices by hardware), the device that generates the clock is the master. The remaining devices that synchronize to the master's clock are called slaves.

Masters within DAQ units in the figure from section 1.1 are labeled red.

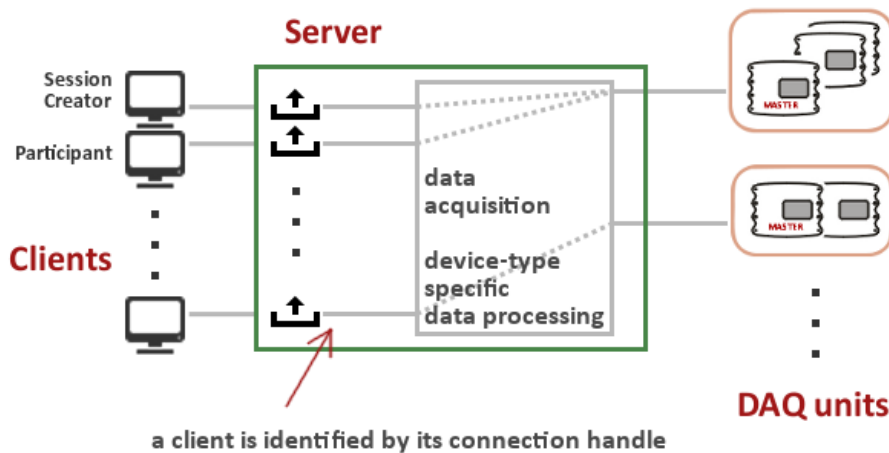
1.1.3 SERVER/CLIENT

The server service runs on the computer that is connected to all devices that data should be acquired from. The server acquires data from its connected devices and distributes/streams it to the clients.

Clients are applications rather than computers that communicate with the server over the network interface (using the network API) to configure and retrieve data from some of the devices that are connected to the server. Multiple clients can run on the same machine, even on the same machine that runs the server service.

The server provides the acquired data for each client in separate buffers which can be imagined as separate mail boxes. The client has to request and retrieve data periodically from its dedicated buffer. Each of those buffers is designed to store about 10 seconds of raw data. If the client doesn't request enough data fast enough, its dedicated buffer fills up and an overflow may be reported to the client when the buffer is full.

The following figure shows the internal data acquisition management and 'routing' of clients to DAQ units.



1.1.4 DATA ACQUISITION UNIT (DAQ UNIT)

A data acquisition unit is a virtual unit and part of the server. It encapsulates one (simplest case) or more hardware-synchronized devices and acquires data from them. The devices that belong to the same DAQ unit have to be of the same device type (and that type has to facilitate hardware synchronization). Therefore, a DAQ unit can also be regarded as a (huge) single device of the actual device type.

For each particular independent group of hardware-synchronized devices, a separate data acquisition unit exists. A physical device can only be part of a single DAQ unit.

Consider an example where six g.USBamp devices are connected to the server. The first four are hardware-synchronized and pooled within DAQ unit 1. The remaining two devices form DAQ unit 2 the same way. Thus, there are two independent groups of devices, and the server will run two independent data acquisition units (DAQ 1 + 2), one for each group.

1.1.5 DATA ACQUISITION SESSION (DAQ SESSION)

A data acquisition session basically refers to a particular data acquisition unit, but in a more conceptual way.

The server itself does not know which of its connected devices are hardware-synchronized. When a client establishes a connection to the server, it must specify a particular group of devices it wants to acquire data from. All devices specified in this group must be hardware-synchronized. If none of these devices are already part of another data acquisition session/unit, the server creates a data acquisition unit for that group. Each client that accesses this particular data acquisition unit is involved in the same data acquisition session.

As soon as a client connects to the server, a unique connection handle is generated, which is used to address the client by the server.

1.1.6 SESSION CREATOR/PARTICIPANT

Each client can access and acquire data from exactly one DAQ unit. For another client, it is possible to participate in a running DAQ session and acquire data from its assigned devices as well (see figure from section 1.1.3), or it can create a new data acquisition session using devices that are not part of an existing DAQ unit yet.

The first client that opens a particular group of devices is the creator of that particular DAQ session. The server creates the corresponding DAQ unit. Session creators have full control over their assigned devices and can configure and operate them.

Any other client that wants to open the same group of devices becomes a participant of that existing DAQ session. The server associates the client with the already existing DAQ unit rather than creating a new one. Participants are not allowed to control data acquisition or configure devices. Basically, they can only retrieve data and other information about the unit.

The lifetime of a DAQ unit expires when the creator closes the connection and the unit gets destroyed by the server. Possible existing participants are notified. The hardware devices from within the unit get released and can be used for subsequent data acquisition sessions. If, on the other hand, a participant closes its connection to the server, the data acquisition unit remains running, at least for the creator and all remaining participants of that session.

1.1.7 CONNECTION HANDLE

A connection handle is basically a unique number and identifies the client and its associated data acquisition session. It is assigned to the client when it creates or opens a data acquisition session. It is important to point out that the creator and participants of the same session get assigned different connection handles.

Whenever a client sends a command to the server, its connection handle is included in the message so the server knows which client (and DAQ session) the command belongs to.

1.1.8 GDS

GDS is the name of the server service itself and stands for g.tec Device Service. This term may occur throughout the Client API reference.

1.2 USING THE C CLIENT API LIBRARIES

g.NEEDaccess comes with APIs for the C programming language and the .NET framework, as well as the documentation of the underlying network API to access g.NEEDaccess Server from platforms other than Windows. This document focuses on the C API libraries for g.NEEDaccess. Separate documentation exists for the remaining APIs.

The C API can use two different ways of communicating with the server. The client-side C API uses the network to communicate with the server. The server-side C API applies the commands directly onto the server at the same machine without using network communication. Both share the same interface. See section *Link against entry point information (lib)* on how to select the API to use. If applications utilize high-performance data acquisition with strict timing constraints, the server-side C API may suit better to meet their requirements.

1.2.1 INCLUDE HEADER FILES

The C API comes with the following header files:

GDSCClientAPI.h	The header base.
GDSCClientAPI_gUSBamp.h	The header file to include when working with g.USBamp devices.
GDSCClientAPI_gHIamp.h	The header file to include when working with g.HIamp devices.

<code>GDSCClientAPI_gNautilus.h</code>	The header file to include when working with g.Nautilus devices.
--	--

For each device, a separate header file exists that contain device-specific functions and the configuration structures for that device. They all include the basic header file `GDSCClientAPI.h`. It is sufficient to include the header file of the device you're working with in your project only, but ensure that the basic header file resides in the same directory as the included header file. If you're working with different types of devices, include the header files for each of those types. DO NOT deploy the header files together with your application.

1.2.2 LINK AGAINST ENTRY POINT INFORMATION (LIB)

For each supported platform, a subfolder in the C API's installation path contains the entry point information lib file to feed the linker with. Currently, the C API supports 64-bit Windows platforms (x64) only.

You can choose to use the client-side C API or the server-side C API by linking against one of the following lib files:

<code>GDSCClientAPI.lib</code>	The client-side C API. Link against this one to communicate with the server over the network (or the local loopback adapter address).
<code>GDSServer.lib</code>	The server-side C API. Link against this one to apply the commands directly onto the server at the same machine without network communication.



IMPORTANT

Do not link against both libraries at the same time. These lib files are NOT static libraries. They contain the entry point information into the dynamic link library (DLL) required for execution. DO NOT deploy the lib file with your application.

1.2.3 EXECUTE WITH DYNAMIC LINK LIBRARIES (DLLS)

The platform subfolder that contains the entry point information lib file also contains the dynamic link libraries (DLLs) that are required for execution. All of these DLLs must reside in the same directory as the application that uses them, or on a global system search path.

If the same machine that runs the application also runs g.NEEDaccess Server, these libraries already reside on a system search path that the system's PATH environment variable points to, and don't need to be deployed additionally.

1.3 GENERAL DATA ACQUISITION PROCEDURE

To successfully acquire data using the Client API methods, ensure that the server is up and running first. Then, perform the following steps in order:

- Initialize the library once using [GDS Initialize](#) before it is used the first time.
- Determine local and destination IP addresses as well as port numbers.
The server, referred to as the destination endpoint, uses port 50223 for incoming connections by default.
It is recommended to run the client application and the server on the same machine. In this case, use the loop-back address (127.0.0.1) for both the local and destination endpoints.
- Initialize two instances of the [GDS_ENDPOINT](#) structure: one for the local (client) and one for the destination (server) endpoints.
- Connect to server and create a connection handle using [GDS Connect](#). Pass the endpoint of the server as the destination endpoint.
- Set callbacks (this is not essential, but strongly recommended).
- Create an instance of the appropriate configuration structure for each device and, if necessary, set the configuration or do other device specific tasks.
- Determine buffer sizes using [GDS GetDataInfo](#) and allocate memory accordingly.
Create and open a file (output stream) if desired, to save data to a file.
- Start acquisition using [GDS StartAcquisition](#).
- Start streaming using [GDS StartStreaming](#).
- Collect data within a loop using [GDS GetData](#).
- Stop streaming using [GDS StopStreaming](#).
- Stop acquisition using [GDS StopAcquisition](#).
- Release previously allocated resources.
Close output file if previously opened.
- Disconnect from server using [GDS Disconnect](#).
- Uninitialize the library using [GDS Uninitialize](#) if it is not used anymore.

1.4 THE GDSCLIENTAPIDEMO PROJECT

GDSClientAPIDemo is a console application illustrating the basic usage of g.NEEDaccess Client API in C++ programming language under Microsoft Windows operating systems. Relevant source code and project files are located in the *GDSClientAPIDemo* folder, which is installed as an example for MS Windows.

Separate examples are provided for using g.NEEDaccess with g.USBamp, g.Hlamp, g.Nautilus PRO, and g.Nautilus devices. A running instance of g.NEEDaccess Server on a host computer running Microsoft Windows OS is required to run the examples. It is assumed that the examples are executed at the same machine that runs the server. The examples use the loopback adapter address for communication with the server (see the g.NEEDaccess Server documentation for details). Furthermore, Microsoft Visual C++ (part of Microsoft Visual Studio) is required to open the preconfigured project files and compile the examples' source code.

1.4.1 COMPILING THE GDSCLIENTAPIDEMO PROJECT

Open one of the MS Visual C++ project files provided in the *GDSClientAPIDemo* folder for each device type (e.g. *gUSBampDemo.vcxproj*). Ensure that you have selected the appropriate platform setting (Win32 for 32-bit, x64 for 64-bit) using the *Configuration Manager* from the

Build menu, and compile it by selecting *Build Solution* from the *Build* menu. Currently, only 64-bit platforms (x64) are supported.

A folder named according to the selected platform and configuration will be created in the `GDSCliEntAPIDemo` folder, and the resulting executable will be built to it. Run the executable either from Visual Studio, by double-clicking it, or from the command line.

On startup, the application scans for supported devices connected to the current machine and prompts you to choose one of them from the presented list. The measurement procedure starts right after selecting a device and records data to a binary file (`data.bin`). Data acquisition will stop automatically when the preconfigured acquisition duration has lapsed.

The configurations of the application and the device are printed to the console window. They're important for reading the recorded data file.

1.4.2 READING THE RECORDED DATA FILE

The recorded data file contains acquired samples as 32-bit float values. Data is organized as a concatenation of scans. Each scan consists of exactly one sample for each acquired channel, starting with the first acquired channel. All samples of a scan are sampled at the same point in time. It can be thought of a two-dimensional matrix in the format `channels x samples`.

The following piece of code illustrates how to read in the recorded data file in MATLAB. The `filename` variable must be set to the name of the recorded data file beforehand. The `channels` variable must be set to the number of acquired channels that the file contains beforehand, including the possibly recorded trigger channel (or other available additional channels). The `data` variable receives the recorded data from the file as a matrix in the format `channels x samples`.

```
filename = 'data.bin';
channels = 16;
fid = fopen(filename, 'rb');
data = fread(fid, [channels, inf], 'float32');
fclose(fid);
```

1.5 THE GDSMULTICLIENTDEMO PROJECT

GDSCliEntDemo is a console application illustrating parallel data acquisition from multiple independent amplifiers concurrently with g.NEEDaccess Client API in C++ programming language under Microsoft Windows operating systems. Relevant source code and project files are located in the `GDSCliEntDemo` folder, which is installed as an example for MS Windows.

Two g.Hlamp devices and a running instance of g.NEEDaccess Server on a host computer running Microsoft Windows OS is required to run the example. It is assumed that the examples are executed at the same machine that runs the server. The examples use the loopback adapter address for communication with the server (see the g.NEEDaccess Server documentation for details). Furthermore, Microsoft Visual C++ (part of Microsoft Visual Studio) is required to open the preconfigured project files and compile the examples' source code.

1.5.1 COMPILING THE GDSMULTICLIENTDEMO PROJECT

Open the `GDSCliEntDemo.vcxproj` MS Visual C++ project file provided in the `GDSCliEntDemo` folder. Adapt the `GDSCliEntDemo.cpp` source code file with the serial numbers of the amplifiers used (in the `deviceSerialsArray` variable around line 25). The specified amplifiers must be connected to the current machine.

Ensure that you have selected the appropriate platform setting (Win32 for 32-bit, x64 for 64-bit) using the *Configuration Manager* from the *Build* menu, and compile it by selecting *Build Solution* from the *Build* menu. Currently, only 64-bit platforms (x64) are supported.

A folder named according to the selected platform and configuration will be created in the GDSCClientAPIDemo folder, and the resulting executable will be built to it. Run the executable either from Visual Studio, by double-clicking it, or from the command line.

On startup, the application opens the specified devices connected to the current machine and acquires 60 seconds of data from their first 40 channels of each of them in parallel. Data acquisition will stop automatically when the preconfigured acquisition duration has lapsed.

Please note that the amplifiers are not hardware-synchronized. Acquisition runs concurrently in two independent threads, each of them writing the data into a separate file for each amplifier (data_0.bin and data_1.bin). This demo does not synchronize the two amplifiers in any way.

1.5.2 READING THE RECORDED DATA FILE

The recorded data file contains acquired samples as 32-bit float values. Data is organized as a concatenation of scans. Each scan consists of exactly one sample for each acquired channel, starting with the first acquired channel. All samples of a scan are sampled at the same point in time. It can be thought of a two-dimensional matrix in the format `channels x samples`.

The following piece of code illustrates how to read in the recorded data file in MATLAB. The `channels` variable must be set to the number of acquired channels that the file contains beforehand, including the possibly recorded trigger channel (or other available additional channels). The `data` variable receives the recorded data from the file as a matrix in the format `channels x samples`.

```
channels = 40;
fid = fopen('data_0.bin', 'rb');
data0 = fread(fid, [channels, inf], 'float32');
fclose(fid);
fid = fopen('data_1.bin', 'rb');
data1 = fread(fid, [channels, inf], 'float32');
fclose(fid);
```

2 CLIENT API – C REFERENCE

2.1 COMMON FUNCTIONALITY (GDSCLIENTAPI.H)

The methods and symbols described in this section only apply to all supported device and source types.

2.1.1 CONSTANTS

ERROR_MESSAGE_LENGTH_MAX

The maximum number of characters an error message can contain, including the terminating null character.

DEVICE_NAME_LENGTH_MAX

The maximum number of characters a device name can contain, including the terminating null character.

IP_ADDRESS_LENGTH_MAX

The maximum number of characters an IP address can contain, including the terminating null character.

2.1.2 ERROR CODES

GDS_ERROR_SUCCESS

The operation completed successfully. No error has occurred.

GDS_ERROR_INVALID_PARAMETER

One of the specified parameters does not contain a valid value.

GDS_ERROR_INVALID_HANDLE

The specified connection handle is invalid.

GDS_ERROR_HANDLE_NOT_ASSOCIATED

The specified connection handle is not associated with any existing data acquisition session created or opened with [GDS Connect](#).

GDS_ERROR_HANDLE_ALREADY_ASSOCIATED

The specified connection handle is already associated with an existing data acquisition session.

GDS_ERROR_ACCESS_DENIED

An existing data acquisition session couldn't be opened because it has been opened exclusively by its creator, or the specified connection handle is not the creator of its associated data acquisition session, or the data acquisition session associated with the specified connection handle already has a creator.

GDS_ERROR_INVALID_DEVICE_LIST

The specified list of devices contains no or invalid entries.

GDS_ERROR_INVALID_DEVICE

A device cannot be part of the same data acquisition session than the others due to an incompatible device type or settings, for example, or the specified device is not part of the specified data acquisition session.

GDS_ERROR_OPEN_DEVICE

A device could not be opened.

GDS_ERROR_DEVICE_ALREADY_IN_USE

A device is already part of an existing data acquisition session with a different list of associated devices, or an attempt was made to open an already existing data acquisition session exclusively.

GDS_ERROR_INVALID_CONFIGURATION

A device could not be configured with the specified configuration, or is not configured at all.

GDS_ERROR_INCOMPATIBLE_CONFIGURATIONS

The device configurations are not compatible among themselves with respect to the associated data acquisition session.

GDS_ERROR_READ_CONFIGURATION

The configuration for a device could not be retrieved.

GDS_ERROR_INVALID_ACQUISITION_STATE

The operation is not allowed during the current acquisition state.

GDS_ERROR_INVALID_STREAMING_STATE

The operation is not allowed during the current streaming state.

GDS_ERROR_BUFFER_TOO_SMALL

The specified number of elements to read exceeds the specified destination buffer's size.

GDS_ERROR_ACQUISITION_BUFFER_OVERFLOW

An overflow occurred in the internal acquisition buffer of a data acquisition session. The server did not fetch data from the acquisition unit within a reasonable time. This indicates a problem on the server. The buffer will be reset automatically, and the client can continue data acquisition.

GDS_ERROR_TRANSMISSION_BUFFER_OVERFLOW

An overflow occurred in a client's streaming buffer on the server. The client did not fetch data from the server in a reasonable time. The buffer will be reset automatically, the client can continue data acquisition.

GDS_ERROR_ACQUISITION_DIED

An internal error occurred during a data acquisition session on the server, forcing data acquisition for that session to stop. Connections are kept alive, the creator of the concerned session can start data acquisition again with [GDS_StartAcquisition](#).

GDS_ERROR_SERIALIZATION

A message or command could not be sent to the server because its content couldn't be serialized on the client side or deserialized. This indicates an internal error on the client or server side.

GDS_ERROR_INVALID_ENDPOINT

The specified endpoint does not exist, or no server is running there.

GDS_ERROR_INVALID_REQUEST_STATUS

The status of the request is invalid.

GDS_ERROR_REQUEST_TIMED_OUT

The server did not reply to the sent request within a reasonable time.

GDS_ERROR_UNKNOWN

An unknown error occurred.

2.1.3 TYPE DEFINITIONS

NULL

The null pointer.

BOOL

The boolean data type, whose values can be [TRUE](#) or [FALSE](#).

FALSE

The *FALSE* value for the [BOOL](#) type.

TRUE

The *TRUE* value for the [BOOL](#) type.

GDS_HANDLE

A handle to a client that is connected with the server.

2.1.4 ENUMERATIONS

GDS_DEVICE_TYPE

An enumeration with supported device types.

GDS_DEVICE_TYPE_NOT_SUPPORTED	The device is of a type that is not supported.
GDS_DEVICE_TYPE_GUSBAMP	A g.USBamp device.
GDS_DEVICE_TYPE_GHIAMP	A g.Hlamp device.
GDS_DEVICE_TYPE_GNAUTILUS	A g.Nautilus device.

GDS_FILTER_TYPE

Types of filters.

GDS_FILTER_TYPE_CHEBYSHEV	A Chebychev filter.
GDS_FILTER_TYPE_BUTTERWORTH	A Butterworth filter.
GDS_FILTER_TYPE_BESSEL	A Bessel filter.

GDS_CHANNEL_DIRECTION

Possible directions of a channel.

GDS_CHANNEL_DIRECTION_IN	The channel is an input channel only.
GDS_CHANNEL_DIRECTION_OUT	The channel is an output channel.

GDS_ELECTRODE_TYPE

Types of electrodes.

GDS_ELECTRODE_TYPE_PASSIVE	A passive electrode.
GDS_ELECTRODE_TYPE_ACTIVE	An active electrode.

2.1.5 FUNCTION POINTERS

GDS_Callback

The type of functions that handles callbacks without an explicit result.

```
typedef void (*GDS_Callback)(GDS\_HANDLE connectionHandle, void* userData)
```

Parameters

<code>connectionHandle</code>	The handle associated with the session that triggered the callback.
<code>userData</code>	The void* can be used to pass data from the user to the callback.

GDS_ResultCallback

The type of functions that handles callbacks that receive a result.

```
typedef void (*GDS_ResultCallback)(GDS\_HANDLE connectionHandle, GDS\_RESULT result, void* userData)
```

Parameters

<code>connectionHandle</code>	The handle associated with the session that triggered the callback.
<code>result</code>	The result of the operation that triggered the callback.
<code>userData</code>	The void* can be used to pass data from the user to the callback.

2.1.6 STRUCTURES

GDS_RESULT

Contains information about the success or failure of a GDS API method call.

Public Fields

- `uint32_t` **ErrorCode**
The error code of the error that occurred, or [GDS_ERROR_SUCCESS](#) on success.
- `char` **ErrorMessage**[\[ERROR_MESSAGE_LENGTH_MAX\]](#)
A message describing the error, including the terminating null character.

GDS_ENDPOINT

Identifies a network endpoint.

Public Fields

- `char` **IpAddress**[\[IP_ADDRESS_LENGTH_MAX\]](#)
The v4 IP address of the endpoint, including the terminating null character.
- `uint16_t` **Port**
The port number of the endpoint.

GDS_DEVICE_INFO

Holds information about a single connected device.

Public Fields

- `char` **Name**[\[DEVICE_NAME_LENGTH_MAX\]](#)
The name of the device.
- [GDS_DEVICE_TYPE](#) **DeviceType**
The type of the device.

GDS_DEVICE_CONNECTION_INFO

Holds information about the devices that belong to a single data acquisition session or no session at all.

Public Fields

- [BOOL](#) **InUse**
Indicates whether the associated devices in *ConnectedDevices* are in use by a data acquisition unit instance. [TRUE](#) indicates that the devices are connected to the computer and owned by a data acquisition unit instance, i.e. at least one client has an open connection to these devices and is the creator. [FALSE](#) indicates that the devices are connected to the computer but not owned by any data acquisition unit instance, i.e. no connection is opened to them.
- `size_t` **ConnectedDevicesLength**
The number of elements in the *ConnectedDevices* array.
- [GDS_DEVICE_INFO](#) ***ConnectedDevices**
An array of connected devices that are in use by a single data acquisition unit instance or that are not in use at all, indicated by the value of *InUse*, of length *ConnectedDevicesLength*.

GDS_CONFIGURATION_BASE

The configuration of the corresponding device type.

Public Fields

- [GDS_DEVICE_INFO](#) **DeviceInfo**
The device that this configuration belongs to.
- void* **Configuration**
A pointer to the actual configuration structure that belongs to the device. The actual type of configuration structure that *Configuration* points to depends on the device type specified in the [GDS_DEVICE_INFO::DeviceType](#) field of the *DeviceInfo* structure.

GDS_FILTER_INFO

Describes the properties of a specific filter.

Public Fields

- [GDS_FILTER_TYPE](#) **TypeId**
The type of the filter.
- uint32_t **Order**
The order of the filter.
- double **SamplingRate**
The sampling rate that the filter is designed for in Hertz.
- double **LowerCutoffFrequency**
The lower cutoff frequency of the filter in Hertz.
- double **UpperCutoffFrequency**
The upper cutoff frequency of the filter in Hertz.

2.1.7 FUNCTIONS

GDS_Initialize

Initializes the GDS library.

void GDS_Initialize()

Remarks

[GDS_Initialize](#) must be called by a process before it uses any of the library functions. When a process is finished using the library, it must call [GDS_Uninitialize](#) before the library is unloaded.

GDS_Uninitialize

Uninitializes the GDS library.

```
void GDS_Uninitialize()
```

Remarks

[GDS_Uninitialize](#) must be called before a process unloads the library. No library function must be called after calling [GDS_Uninitialize](#).

GDS_GetConnectedDevices

Gets a list of devices connected to the GDS server at the specified destination endpoint.

[GDS_RESULT](#) GDS_GetConnectedDevices([GDS_ENDPOINT](#) destinationEndpoint, [GDS_ENDPOINT](#) localEndpoint, [GDS_DEVICE_CONNECTION_INFO](#) **connectedDevices, size_t *connectedDevicesCount)

Parameters	
destinationEndpoint	The endpoint of the server that should return information about the devices connected to it.
localEndpoint	The local endpoint of the client that receives the reply from the server.
connectedDevices	A pointer to a variable that receives a pointer to an array of GDS_DEVICE_CONNECTION_INFO structures containing information about connected devices on the server.
connectedDevicesCount	A pointer to a variable that receives the number of elements in the returned <i>connectedDevices</i> array.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_ENDPOINT	Couldn't retrieve information about connected devices from the specified endpoint. Maybe it doesn't exist, or no server is running there, or a firewall blocked the request.

Remarks

The method allocates memory for the *connectedDevices* array. It's up to the caller to release that memory afterwards using [GDS_FreeConnectedDevicesList](#).

GDS_FreeConnectedDevicesList

Frees memory allocated by [GDS_GetConnectedDevices](#).

[GDS_RESULT](#) GDS_FreeConnectedDevicesList([GDS_DEVICE_CONNECTION_INFO](#)
**connectedDevices, size_t connectedDevicesCount)

Parameters	
connectedDevices	The pointer to the array of connected devices retrieved with GDS_GetConnectedDevices .
connectedDevicesCount	The number of elements in the array that <i>connectedDevices</i> points to.
Returns	
A GDS_RESULT structure containing information about the operation's success. It may represent one of the following errors:	
GDS_ERROR_SUCCESS	The operation succeeded.

GDS_Connect

Establishes a connection to the server and creates a new or opens an existing data acquisition session with the devices with the specified names.

[GDS_RESULT](#) GDS_Connect([GDS_ENDPOINT](#) destinationEndpoint, [GDS_ENDPOINT](#) localEndpoint, char (*deviceNames)[[DEVICE_NAME_LENGTH_MAX](#)], size_t deviceNamesCount, [BOOL](#) openExclusively, [GDS_HANDLE](#) *connectionHandle, [BOOL](#) *isCreator)

Parameters	
destinationEndpoint	The remote endpoint address of the server that the client should connect with.
localEndpoint	The local endpoint address of the client that receives replies from the server.
deviceNames	The list of device names identifying the devices that are connected to the server and part of the data acquisition to open or to create. Ordering matters. The first element in the array is considered as the master device if multiple devices are hardware-synchronized. The others are considered as slave devices in the specified order.
deviceNamesCount	The number of device names in the <i>deviceNames</i> array.
openExclusively	TRUE to prevent a newly created data acquisition session from being opened or accessed by a second call to GDS_Connect with the same <i>deviceNames</i> list; otherwise, FALSE .
connectionHandle	A pointer to a variable that receives the connection handle issued to the caller by the server. This connection handle is associated with the opened data acquisition session until GDS_Disconnect is called for that handle, or until the creator closes the data acquisition session.
isCreator	A pointer to a variable that receives a value indicating whether the caller is the creator or a participant of the data acquisition session.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_ENDPOINT	Couldn't retrieve information about connected devices from the specified endpoint. Maybe it doesn't exist, or no server is running there, or a firewall blocked the request.
GDS_ERROR_INVALID_PARAMETER	<i>connectionHandle</i> is NULL , or <i>isCreator</i> is NULL .
GDS_ERROR_INVALID_DEVICE_LIST	<i>deviceNames</i> is NULL , or <i>deviceNamesCount</i> equals zero.
GDS_ERROR_INVALID_DEVICE	A device cannot be part of the same data acquisition session than the first device due to incompatible device type or settings, for example, or an existing data acquisition session should be opened, but an additional device was specified that is not part of that session.
GDS_ERROR_DEVICE_ALREADY_IN_USE	A device is already part of an existing data acquisition session with a different list of associated devices, or an attempt was made to open an already existing data acquisition session exclusively (i.e. <i>openExclusively</i> is set to TRUE).
GDS_ERROR_ACCESS_DENIED	An existing data acquisition session couldn't be opened because it has been opened exclusively by its creator.
GDS_ERROR_OPEN_DEVICE	At least one device in <i>deviceNames</i> refers to a device that couldn't be found or opened.

Remarks

A data acquisition session is identified by the list of device names that are part of the session. Each successful call of [GDS_Connect](#) receives a unique connection handle. The connection handle is used to identify a creator or participant of a data acquisition session and must be specified in almost every method of the API. Multiple connection handles can refer to the same data acquisition session that are currently existing on a particular server.

Once [GDS_Connect](#) is called for a specific list of device names the first time, a new data acquisition session is created for those devices, and data is acquired synchronously for all devices in that list. The returned connection handle is then referred to as the creator of the newly created data acquisition session.

If [GDS_Connect](#) is called with the exact same list of devices names that have already been opened by a previous call of [GDS_Connect](#), and that previous call did not open the devices exclusively (i.e. *openExclusively* was set to [FALSE](#)), [GDS_Connect](#) succeeds, and returns [FALSE](#) in the variable pointed to by *isCreator*. The returned connection handle is then referred to as the participant of the existing data acquisition session. [GDS_Connect](#) will fail if an existing data acquisition session should be opened, but *deviceNames* is not exactly equal to the list of device names that was specified for that existing session when it was created (i.e. when [GDS_Connect](#) was called the first time for those device names).

Use [GDS Disconnect](#) to release the connection. All open connections must be released before unloading the library.

GDS_Disconnect

Dissociates the specified connection handle from its associated data acquisition session and closes the connection to the server.

[GDS_RESULT](#) GDS_Disconnect([GDS_HANDLE](#) *connectionHandle)

Parameters	
connectionHandle	A pointer to the connection handle to dissociate and disconnect.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_INVALID_PARAMETER	<i>connectionHandle</i> is NULL .
GDS_ERROR_INVALID_HANDLE	The value that <i>connectionHandle</i> points to is invalid (i.e. equals NULL).
GDS_ERROR_HANDLE_NOT_ASSOCIATED	The connection handle that <i>connectionHandle</i> points to is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is currently running for the specified connection handle.

Remarks

All registered callbacks for the specified connection handle will be cleared automatically. If *connectionHandle* was the creator of its associated data acquisition session, the whole data acquisition session will be destroyed, all associated clients get disconnected and their **SessionCreatorDied** callback is executed if set. Otherwise, only the specified connection handle gets dissociated, i.e. the data acquisition session remains running for all remaining connection handles that are still associated with it, and for which [GDS_Disconnect](#) has not been called before.



IMPORTANT

[GDS_Disconnect](#) must be called for each connection handle opened with [GDS_Connect](#) before the library is unloaded. [GDS_Disconnect](#) must not be called from a process while that process is unloading the client API library because [GDS_Disconnect](#) is waiting for threads to complete, which would prevent the library from getting unloaded. This scenario may happen if [GDS_Disconnect](#) is called from a static (or singleton) class's destructor that is only called during library unloading.

The operating system uses a lock for both unloading libraries and detaching threads, which means that waiting for completion of a thread during unloading of the library will result in a

deadlock (the lock cannot be acquired for detaching the thread because it has already been acquired by the unloading, but unloading cannot complete while the thread has not been detached).

GDS_BecomeCreator

Elevates the specified connection handle to the creator of its associated data acquisition session.

[GDS_RESULT](#) GDS_BecomeCreator([GDS_HANDLE](#) participantConnectionHandle)

Parameters	
participantConnectionHandle	The connection handle of the participant that wants to get elevated to the creator of its associated data acquisition session.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>participantConnectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>participantConnectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_ACCESS_DENIED	The data acquisition session associated with <i>participantConnectionHandle</i> does already have an active creator.

Remarks

This method can be called for any participant of a data acquisition session in case the creator of that session has died unexpectedly. Otherwise, this method will fail.

See [GDS_Connect](#) for details on the terms 'creator' and 'participant' of a data acquisition session.

The **NewSessionCreator** callback is raised for all participants of the associated data acquisition session on success.

GDS_BecomeParticipant

Downgrades the specified connection handle from the creator of its associated data acquisition session to a simple participant.

[GDS_RESULT](#) GDS_BecomeParticipant([GDS_HANDLE](#) creatorConnectionHandle)

Parameters	
creatorConnectionHandle	The connection handle of the creator that wants to get downgraded to a participant of its associated data acquisition session

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>creatorConnectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>creatorConnectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_ACCESS_DENIED	<i>creatorConnectionHandle</i> is not creator of its associated data acquisition session.

Remarks

This method can only be called for creators of data acquisition sessions. Otherwise, this method will fail.

See [GDS_Connect](#) for details on the terms 'creator' and 'participant' of a data acquisition session.

The **SessionCreatorDied** callback is raised for all participants of the associated data acquisition session on success.

GDS_SetConfiguration

Sets the configuration for each device of the data acquisition session that was created with the specified connection handle.

[GDS_RESULT](#) GDS_SetConfiguration([GDS_HANDLE](#) connectionHandle,
[GDS_CONFIGURATION_BASE](#) *deviceConfigurations, size_t deviceConfigurationsCount)

Parameters

connectionHandle	The connection handle that the devices of its associated data acquisition session to configure has been created with.
deviceConfigurations	An array of configuration structures where each element holds the configuration for the corresponding device in the associated data acquisition session.
deviceConfigurationsCount	The number of elements in deviceConfigurations, i.e. the number of devices.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_ACCESS_DENIED	<i>connectionHandle</i> is not creator of its associated data acquisition session.
GDS_ERROR_INVALID_DEVICE_LIST	The specified list of device names does not match the list that was specified for <i>connectionHandle</i> on creation.
GDS_ERROR_INVALID_CONFIGURATION	At least one device could not be configured with the specified configuration due to some invalid parameters or invalid combinations of parameters in the corresponding configuration structure.
GDS_ERROR_INCOMPATIBLE_CONFIGURATIONS	The device configurations are not compatible among themselves with respect to the associated data acquisition session.
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is currently running.

Remarks

The *deviceConfigurations* array holds a configuration for each device specified in the previous call of [GDS Connect](#) for the associated data acquisition session, in the order they were specified in [GDS Connect](#). The device configuration structure at index *l* in the *deviceConfigurations* array holds the configuration for the device specified at index *l* in the *deviceNames* parameter of the corresponding call to [GDS Connect](#).

Each call of [GDS SetConfiguration](#) must always contain one configuration for each device of the associated data acquisition unit.

[GDS SetConfiguration](#) can only be called for connection handles that are creator of their associated data acquisition session (see [GDS Connect](#) for details on who is creator of a data acquisition session).

GDS_GetConfiguration

Gets the current configuration for each device of the associated data acquisition session.

[GDS_RESULT](#) GDS_GetConfiguration([GDS_HANDLE](#) connectionHandle,
[GDS_CONFIGURATION_BASE](#) **deviceConfigurations, size_t *deviceConfigurationsCount)

Parameters

connectionHandle	The connection handle to return the configuration for the devices of its associated data acquisition session for.
deviceConfigurations	A pointer to a variable that receives a pointer to the array of device configuration structures.
deviceConfigurationsCount	A pointer to a variable that receives the number of elements in the array that <i>deviceConfigurations</i> points to, i.e. the number of devices.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_READ_CONFIGURATION	The configuration for at least one device of the associated data acquisition session could not be retrieved.

Remarks

The *deviceConfigurations* array receives the configuration for each device specified in the previous call of [GDS_Connect](#) for the associated data acquisition session, in the order they were specified in [GDS_Connect](#). The device configuration structure at index *i* in the *deviceConfigurations* array receives the configuration for the device specified at index *i* in the *deviceNames* parameter of the corresponding call to [GDS_Connect](#).

The method allocates memory for the device configuration structures. It's up to the caller to release that memory afterwards using [GDS_FreeConfigurationList](#).

[GDS_GetConfiguration](#) can be called for any connection handle that is associated with a specific data acquisition session.

GDS_FreeConfigurationList

Frees memory allocated by [GDS_GetConfiguration](#).

[GDS_RESULT](#) GDS_FreeConfigurationList([GDS_CONFIGURATION_BASE](#) **deviceConfigurations, size_t deviceConfigurationsCount)

Parameters	
deviceConfigurations	The pointer to the device configuration structure array retrieved with GDS_GetConfiguration .
deviceConfigurationsCount	The number of elements in the array that <i>deviceConfigurations</i> points to.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
-----------------------------------	--------------------------

GDS_StartAcquisition

Starts data acquisition on all devices of the data acquisition session that was created with the specified connection handle.

[GDS_RESULT](#) GDS_StartAcquisition([GDS_HANDLE](#) connectionHandle)

Parameters	
connectionHandle	The connection handle that the data acquisition session to start has been created with.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_ACCESS_DENIED	<i>connectionHandle</i> is not creator of its associated data acquisition session.
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is already running.
GDS_ERROR_INVALID_CONFIGURATION	The devices of the associated data acquisition session have not been configured yet successfully.

Remarks

Call [GDS_StartStreaming](#) afterwards to retrieve data.

Note that the behavior of [GDS_StartStreaming](#) is different from [GDS_StartAcquisition](#). While [GDS_StartAcquisition](#) actually starts data acquisition on the devices without streaming data to connected connection handles, [GDS_StartStreaming](#) enables streaming of acquired data to the connection associated with the specified connection handle. Hence, [GDS_StartStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [GDS_StartAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [GDS_Connect](#) for details on who is the creator of a data acquisition session).

GDS_StopAcquisition

Stops data acquisition on all devices of the data acquisition session created with the specified connection handle.

[GDS_RESULT](#) GDS_StopAcquisition([GDS_HANDLE](#) connectionHandle)

Parameters	
connectionHandle	The connection handle that the data acquisition session to stop has been created with.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_ACCESS_DENIED	<i>connectionHandle</i> is not creator of its associated data acquisition session.
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is not running.

Remarks

Note that the behavior of [GDS_StopStreaming](#) is different from [GDS_StopAcquisition](#). While [GDS_StopAcquisition](#) actually stops data acquisition on the devices and streaming to all connected connection handles, [GDS_StopStreaming](#) disables streaming of acquired data just for the connection associated with the specified connection handle. Hence, [GDS_StopStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [GDS_StopAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [GDS_Connect](#) for details on who is the creator of a data acquisition session).

GDS_StartStreaming

Enables data streaming for the specified connection handle.

[GDS_RESULT](#) GDS_StartStreaming([GDS_HANDLE](#) connectionHandle)

Parameters	
<i>connectionHandle</i>	The connection handle to start streaming for.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is not running.

Remarks

[GDS_GetData](#) can be called afterwards to retrieve acquired data. *connectionHandle* must be associated with a data acquisition session with a preceding successful call to [GDS_Connect](#). Data acquisition must have been already started by the creator with [GDS_StartAcquisition](#) for that session.

Note that the behavior of [GDS_StartStreaming](#) is different from [GDS_StartAcquisition](#). While [GDS_StartAcquisition](#) actually starts data acquisition on the devices without streaming data to connected connection handles, [GDS_StartStreaming](#) enables streaming of acquired data to the connection associated with the specified connection handle. Hence, [GDS_StartStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [GDS_StartAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [GDS_Connect](#) for details on who is the creator of a data acquisition session).

GDS_StopStreaming

Disables data streaming for the specified connection handle.

[GDS_RESULT](#) GDS_StopStreaming([GDS_HANDLE](#) connectionHandle)

Parameters	
connectionHandle	The connection handle to stop streaming for.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

Data acquisition remains running for connection handles that have streaming still enabled. Note that the behavior of [GDS_StopStreaming](#) is different from [GDS_StopAcquisition](#). While [GDS_StopAcquisition](#) actually stops data acquisition on the devices and streaming to all connected connection handles, [GDS_StopStreaming](#) disables streaming of acquired data just for the connection associated with the specified connection handle. Hence, [GDS_StopStreaming](#) can be called for any connection handle that is associated with a specific data acquisition session, while [GDS_StopAcquisition](#) can only be called for connection handles that are creators of a specific data acquisition session (see [GDS_Connect](#) for details on who is the creator of a data acquisition session).

GDS_GetDataInfo

Returns information about organization of the data to retrieve with [GDS_GetData](#) for the specified connection handle.

[GDS_RESULT](#) GDS_GetDataInfo([GDS_HANDLE](#) connectionHandle, size_t *scanCount, size_t *channelsPerDevice, size_t *channelsPerDeviceCount, size_t *bufferSizeInSamples)

Parameters	
<i>connectionHandle</i>	The connection handle to retrieve information about organization of the data to retrieve.
<i>scanCount</i>	A pointer to a variable that holds the number of scans to calculate the remaining output parameters for, or to a variable with a value of zero to receive the number of currently available scans into that variable and calculate the remaining output parameters for that number of currently available scans.
<i>channelsPerDevice</i>	A pointer to an allocated array that receives the numbers of channels that each device is acquiring, including synchronously sampled digital trigger channels, or NULL to receive the required size of that array. The number of elements in that array is equal to the total number of devices that are part of the data acquisition session that is associated with the specified connection handle. The element at index <i>i</i> corresponds to the device with serial number at index <i>i</i> in the <i>deviceNames</i> list parameter of the GDS_Connect method called for the specified connection handle.
<i>channelsPerDeviceCount</i>	A pointer to a variable that holds the number of elements in the <i>channelsPerDevice</i> array.
<i>bufferSizeInSamples</i>	Receives the number of samples for the specified or returned number of scans. This is the number of elements a buffer must contain to be able to hold that amount of scans.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

<u>GDS_ERROR_SUCCESS</u>	The operation succeeded.
<u>GDS_ERROR_INVALID_PARAMETER</u>	<i>scanCount</i> is <u>NULL</u> , or <i>channelsPerDeviceCount</i> is <u>NULL</u> , or <i>channelsPerDeviceCount</i> is less than the number of associated devices and <i>channelsPerDevice</i> is not <u>NULL</u> , or <i>bufferSizeInSamples</i> is <u>NULL</u> .
<u>GDS_ERROR_INVALID_HANDLE</u>	<i>connectionHandle</i> is invalid.
<u>GDS_ERROR_HANDLE_NOT_ASSOCIATED</u>	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with <u>GDS.Connect</u> .

Remarks

A sample or element is the value of exactly one channel and is a 32-bit floating point number. A scan consists of exactly one sample for each acquired channel. The total number of acquired channels of all devices is the sum over all elements of *channelsPerDevice*, which is the number of elements of a single scan. The required size of the buffer in bytes to be able to hold the specified or returned number of scans is *bufferSizeInSamples* * *sizeof(float)*.

Call this method with *channelsPerDevice* equal to [NULL](#) to retrieve the required size of the *channelsPerDevice* array into *channelsPerDeviceCount*. Then, call this method again with the allocated *channelsPerDevice* array to retrieve its content.

GDS_GetData

Reads the specified number of scans into the specified destination buffer.

[GDS_RESULT](#) GDS_GetData([GDS_HANDLE](#) connectionHandle, size_t *scanCount, float *destinationBuffer, size_t destinationBufferSizeInSamples)

Parameters

connectionHandle	The connection handle to retrieve data for.
scanCount	A pointer to a variable that either holds the number of scans to read. If variable pointed to holds zero, the method reads as many scans as either currently available or the specified destination buffer can contain, whichever is smaller. This variable receives the number of actually read scans when the method returns.
destinationBuffer	The buffer where the read scans are written to.
destinationBufferSizeInSamples	The maximum number of elements that <i>destinationBuffer</i> can contain. See GDS_GetDataInfo for details on the definition of buffer elements.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_PARAMETER	<i>scanCount</i> is NULL , or <i>destinationBuffer</i> is NULL .
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .
GDS_ERROR_BUFFER_TOO_SMALL	The specified number of scans to read exceeds the destination buffer's size.
GDS_ERROR_INVALID_ACQUISITION_STATE	Data acquisition is not running.
GDS_ERROR_INVALID_STREAMING_STATE	Streaming has not been started.

Remarks

Data is retrieved in microvolts as concatenated scans. A scan consists of exactly one sample for each acquired channel. A sample or element is the 32-bit floating point value of exactly one channel in microvolts. The value of sample *s* in channel *c* and a total number of *N* channels is stored in *destinationBuffer[s * N + c]*. The total number of acquired channels of all devices (i.e. the

number of all elements in a single scan) can be determined by calling [GDS GetDataInfo](#) with a *scanCount* value of 1.

This method verifies that the specified destination buffer is large enough to hold data for the specified number of scans using *destinationBufferSizeInSamples*. If there are less scans available than specified by *scanCount*, the method reads only all available scans into the destination buffer, and returns the number of actually read scans in *scanCount*.

Use [GDS GetDataInfo](#) to determine the number of currently available scans and the required destination buffer size. It is recommended to call [GDS GetDataInfo](#) just once before streaming starts (or after the configuration has changed) for a specific amount of scans, allocate a fixed sized buffer and re-use this buffer for every consecutive [GDS GetData](#) call without re-allocating it before each call.



IMPORTANT

If network or loopback adapter address is used to communicate with the server and [GDS GetData](#) is called periodically with a fixed *scanCount* value that is too small (like 1...4 at 256 Hz sampling rate), the [GDS GetData](#) request might be too slow to retrieve the data in realtime and delays or data acquisition errors stating buffer overflows may occur. In this case, it is better to link with the server-side C API that operates directly on the server at the same machine without using network communication.

2.1.8 CALLBACK REGISTRATION FUNCTIONS

GDS_SetConfigurationChangedCallback

Sets or clears the **ConfigurationChanged** callback that is triggered when the creator of a data acquisition session has changed the configuration of the associated devices.

[GDS RESULT](#) GDS_SetConfigurationChangedCallback([GDS HANDLE](#) connectionHandle, [GDS Callback](#) configurationChangedCallback, void* usrData)

Parameters	
<i>connectionHandle</i>	The connection handle to set the ConfigurationChanged callback for.
<i>configurationChangedCallback</i>	The callback method to execute when the creator of a data acquisition session has changed the configuration of the associated devices, or NULL to clear the ConfigurationChanged callback for the specified connection handle.
<i>userData</i>	The user data that should be passed to the callback.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS Connect .

Remarks

When [GDS_SetConfigurationChangedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **ConfigurationChanged** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetDataAcquisitionStartedCallback

Sets or clears the **DataAcquisitionStarted** callback that is triggered when the creator of a data acquisition session has started acquisition.

[GDS_RESULT](#) GDS_SetDataAcquisitionStartedCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) dataAcquisitionStartedCallback, void* userData)

Parameters

connectionHandle	The connection handle to set the DataAcquisitionStarted callback for.
dataAcquisitionStartedCallback	The callback method to execute when the creator of a data acquisition session has started acquisition, or NULL to clear the DataAcquisitionStarted callback for the specified connection handle.
userData	The user data that should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When [GDS_SetDataAcquisitionStartedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataAcquisitionStarted** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetDataAcquisitionStoppedCallback

Sets or clears the **DataAcquisitionStopped** callback that is triggered when the creator of a data acquisition session has stopped acquisition.

[GDS_RESULT](#) GDS_SetDataAcquisitionStoppedCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) dataAcquisitionStoppedCallback, void* userData)

Parameters	
connectionHandle	The connection handle to set the DataAcquisitionStopped callback for.
dataAcquisitionStoppedCallback	The callback method to execute when the creator of a data acquisition session has stopped acquisition, or NULL to clear the DataAcquisitionStopped callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When [GDS_SetDataAcquisitionStoppedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataAcquisitionStopped** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetDataAcquisitionErrorCallback

Sets or clears the **DataAcquisitionError** callback that is triggered when an error occurs during data acquisition.

[GDS_RESULT](#) GDS_SetDataAcquisitionErrorCallback([GDS_HANDLE](#) connectionHandle, [GDS_ResultCallback](#) dataAcquisitionErrorCallback, void* userData)

Parameters	
connectionHandle	The connection handle to set the DataAcquisitionError callback for.
dataAcquisitionErrorCallback	The callback method to execute when an error occurs during data acquisition, or NULL to clear the DataAcquisitionError callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When [GDS_SetDataAcquisitionErrorCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataAcquisitionError** callback for the specified connection handle, set [NULL](#) as the callback method.

The parameter of the callback method receives a [GDS_RESULT](#) structure containing information about the error that has occurred during data acquisition.

GDS_SetDataReadyCallback

Sets or clears the **DataReady** callback that is triggered when the given number of scans is available for the specified connection handle.

[GDS_RESULT](#) GDS_SetDataReadyCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) dataReadyCallback, size_t numberOfScans, void* userData)

Parameters

connectionHandle	The connection handle to set the DataReady callback for.
dataReadyCallback	The callback method to execute when the given number of scans is available, or NULL to clear the DataReady callback for the specified connection handle.
numberOfScans	The minimum number of scans that must be retrieved during data acquisition to trigger the callback.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	connectionHandle is not associated with any existing data acquisition session created or opened with GDS_Connect.
GDS_ERROR_INVALID_PARAMETER	numberOfScans equals zero and dataReadyCallback is not NULL .

Remarks

When [GDS_SetDataReadyCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **DataReady** callback for the specified connection handle, set [NULL](#) as the the callback method.

GDS_SetSessionCreatorDiedCallback

Sets or clears the **SessionCreatorDied** callback that is triggered when the creator of a data acquisition session has died unexpectedly or when [GDS_BecomeParticipant](#) was called successfully.

[GDS_RESULT](#) GDS_SetSessionCreatorDiedCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) sessionCreatorDiedCallback, void* userData)

Parameters	
connectionHandle	The connection handle to set the SessionCreatorDied callback for.
sessionCreatorDiedCallback	The callback method to execute when the creator of a data acquisition session has died unexpectedly or when GDS_BecomeParticipant was called successfully, or NULL to clear the SessionCreatorDied callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When [GDS_SetSessionCreatorDiedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **SessionCreatorDied** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetNewSessionCreatorCallback

Sets or clears the **NewSessionCreator** callback that is triggered when a data acquisition session has a new creator (i.e. [GDS_BecomeCreator](#) was called successfully).

[GDS_RESULT](#) GDS_SetNewSessionCreatorCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) newSessionCreatorCallback, void* userData)

Parameters	
connectionHandle	The connection handle to set the NewSessionCreator callback for.
newSessionCreatorCallback	The callback method to execute when a data acquisition session has got a new creator, or NULL to clear the NewSessionCreator callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When [GDS_SetNewSessionCreatorCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **NewSessionCreator** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetServerDiedCallback

Sets or clears the **ServerDied** callback that is triggered when the server was forced to shut down unexpectedly before all open connections were closed.

[GDS_RESULT](#) GDS_SetServerDiedCallback([GDS_HANDLE](#) connectionHandle, [GDS Callback](#) serverDiedCallback, void* userData)

Parameters	
connectionHandle	The connection handle to set the ServerDied callback for.
serverDiedCallback	The callback method to execute when the server was forced to shut down unexpectedly before all open connections were closed, or NULL to clear the ServerDied callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS Connect .

Remarks

When [GDS_SetServerDiedCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **ServerDied** callback for the specified connection handle, set [NULL](#) as the callback method.

GDS_SetForcedClientShutdownCallback

Sets or clears the **ForcedClientShutdown** callback that is triggered when the server forced the connection associated with the specified connection handle to shut down unexpectedly.

[GDS_RESULT](#) GDS_SetForcedClientShutdownCallback([GDS_HANDLE](#) connectionHandle, [GDS_Callback](#) forcedClientShutdownCallback, void* userData)

Parameters

connectionHandle	The connection handle to set the ForcedClientShutdown callback for.
forcedClientShutdownCallback	The callback method to execute when the server forced the connection associated with the specified connection handle to shut down unexpectedly, or NULL to clear the ForcedClientShutdown callback for the specified connection handle.
userData	The user data which should be passed to the callback.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any existing data acquisition session created or opened with GDS_Connect .

Remarks

When the **ForcedClientShutdown** callback registered for the specified connection handle executes, the server has already terminated the connection, and the connection handle is no longer valid. Any registered callbacks for the connection handle have been cleared automatically. [GDS_Disconnect](#) does not need to be called afterwards, since it will fail anyway. This may occur when the server does not receive heartbeat replies anymore from the endpoint associated with the specified connection handle.

When [GDS_SetForcedClientShutdownCallback](#) has already been called for the specified connection handle, the existing callback method gets replaced by a new one.

To clear the **ForcedClientShutdown** callback for the specified connection handle, set [NULL](#) as the callback method.

2.2 G.USBAMP SPECIFICS (GDSCLIENTAPI_GUSBAMP.H)

The methods and symbols described in this section only apply for g.USBamp devices.

2.2.1 CONSTANTS

GDS_GUSBAMP_CHANNELS_MAX

The maximum number of channels a g.USBamp device can provide.

GDS_GUSBAMP_GROUPS_MAX

The number of groups of a g.USBamp device that can be connected to separate or common ground and reference channels.

GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNELS_MAX

The number of asynchronous digital I/O channels a g.USBamp device provides. g.USBamp version 3.0 devices provide 4 digital output channels. g.USBamp version 2.0 devices provide 2 digital input channels and 2 digital output channels.

GDS_GUSBAMP_DEVICE_INFORMATION_LENGTH_MAX

The maximum number of characters the device info string can contain, including the terminating null character.

2.2.2 ENUMERATIONS

GDS_GUSBAMP_WAVESHAPE

The wave shapes provided by the internal signal generator.

GDS_GUSBAMP_WAVESHAPE_SQUARE	A square wave.
GDS_GUSBAMP_WAVESHAPE_SAWTOOTH	A saw tooth.
GDS_GUSBAMP_WAVESHAPE_SINE	A sine wave.
GDS_GUSBAMP_WAVESHAPE_DRL	Driven Right Leg (DRL).
GDS_GUSBAMP_WAVESHAPE_NOISE	Noise.

2.2.3 STRUCTURES

GDS_GUSBAMP_SAMPLING_RATE_FEATURES

The supported features at a specific sampling rate.

Public Fields

- `uint32_t` **SamplingRate**
The sampling rate that the features of the current instance belong to.
- `uint16_t` **RecommendedNumberOfScans**
The recommended number of scans for *SamplingRate*.

GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNEL

Describes the state of an asynchronous digital I/O channel.

Public Fields

- `unsigned int` **ChannelNumber**
The one-based channel number of the digital I/O channel described by this instance.
- [GDS_CHANNEL_DIRECTION](#) **Direction**
Indicates whether the channel described by this instance is a digital input or output channel.
- [BOOL](#) **Value**
The digital value of the channel.

GDS_GUSBAMP_SCALING

The scaling values of each channel of a single g.USBamp device.

Public Fields

- `float` **Offset**[\[GDS_GUSBAMP_CHANNELS_MAX\]](#)
The offset for each channel in microvolts.
- `float` **ScalingFactor**[\[GDS_GUSBAMP_CHANNELS_MAX\]](#)
The scaling factor for each channel.

GDS_GUSBAMP_SIGNAL_GENERATOR_CONFIGURATION

The configuration of the internal signal generator.

Public Fields

- [BOOL](#) **Enabled**
Indicates whether the internal signal generator should be enabled.
- [GDS_GUSBAMP_WAVE_SHAPE](#) **WaveShape**
The wave shape of the signal generator.
- `uint32_t` **Frequency**
The frequency of the signal generator in hertz. Valid values range from 1 Hz to 100 Hz.
- `int32_t` **Amplitude**
The amplitude of the signal generator in mV. Valid values range from -250 mV to +250 mV.
- `int32_t` **Offset**
The offset of the signal generator in mV. Valid values range from -200 mV to +200 mV.

GDS_GUSBAMP_CHANNEL_CONFIGURATION

The configuration of a single channel.

Public Fields

- [BOOL](#) **Acquire**
Indicates whether the channel that this configuration belongs to should be included in data acquisition.
- `uint32_t` **BipolarChannel**
The bipolar derivation setting for the channel. Set this value to zero if unipolar derivation should be used with ground and reference electrodes that are connected to the ground and reference socket of the group that this channel belongs to. Otherwise, if bipolar derivation should be performed for the channel that this configuration belongs to, set *BipolarChannel* to the one-based channel number whose signal should be subtracted from the measured signal of this channel.
- `int32_t` **BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GDS_GUSBAMP_GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- `int32_t` **NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GDS_GUSBAMP_GetNotchFilters](#). A value of -1 indicates that no notch filter is used for the channel.

GDS_GUSBAMP_CONFIGURATION

The configuration of a g.USBamp device.

Public Fields

- `uint32_t` **SampleRate**
The sample rate of the analog inputs in hertz. Valid values are: 32, 64, 128, 256, 512, 600, 1200, 2400, 4800, 9600, 19200, 38400. Eight-channel g.USBamp devices support the following sample rates only: 128, 256, 512.
- `size_t` **NumberOfScans**
The number of scans to retrieve from the device at once, or zero to determine the recommended number of scans for the specified sampling rate automatically. This value depends on the value of *SampleRate*. The following table lists the recommended values of *NumberOfScans* for all possible sample rates.

Sampling Rate [Hz]	Recommended value for NumberOfScans
32	1
64	2
128	4
256	8
512	16
600	32
1200	64
2400	128
4800	256
9600	512
19200	512
38400	512

- **[BOOL ShortCutEnabled](#)**
Indicates whether the short cut socket is enabled. If short cut is enabled, a HIGH level on the SC input socket of the amplifier disconnects the electrodes from the amplifier input stage as long as the HIGH level is applied. If short cut is disabled, the level on the SC input socket of the amplifier is ignored, and the amplifier always delivers the currently measured values on its input channels.
- **[BOOL CounterEnabled](#)**
Indicates whether a sample counter should be applied on channel 16 instead of the measured signal if selected for acquisition (overrun at 1,000,000 samples).
- **[BOOL TriggerEnabled](#)**
Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition in an additional logical channel that is appended to the synchronously sampled analog input channels.

If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels.

g.USBamp version 2.0 has just one trigger line, so the values of the trigger channel can be 0 (LOW) or 250,000 (HIGH).

In version 3.0, there are 8 trigger lines, coming bit-encoded as UINT8 value on the trigger channel. If all of the 8 trigger lines are HIGH, the value of the channel equals 255.0. If e.g. trigger line 0 to 3 are HIGH, the value of the channel equals 15.0.

- **[GDS GUSBAMP SIGNAL GENERATOR CONFIGURATION](#) InternalSignalGenerator**
The configuration of the internal signal generator.
- **[BOOL CommonGround](#)[\[GDS GUSBAMP GROUPS MAX\]](#)**
The common ground setting for each of the four groups A, B, C, and D. Each element in the array indicates whether the ground socket of the corresponding group (A, B, C, or D) should be connected to common ground. The common ground

setting for group A (channels 1 to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.

A value of zero of an element in the array indicates that the ground socket of the corresponding group is not connected to common ground.

A non-zero element indicates that the ground socket of the corresponding group is connected to common ground.

- **BOOL CommonReference**[\[GDS GUSBAMP GROUPS MAX\]](#)

The common reference setting for each of the four groups A, B, C, and D.

Each element in the array indicates whether the reference socket of the corresponding group (A, B, C, or D) should be connected to common reference. The common reference setting for group A (channels 1 to 4) is represented by the element at index zero, group B (channels 5 to 8) at index 1, group C (channels 9 to 12) at index 2, and group D (channels 13 to 16) at index 3.

A value of zero of an element in the array indicates that the reference socket of the corresponding group is not connected to common reference.

A non-zero element indicates that the reference socket of the corresponding group is connected to common reference.

- **GDS GUSBAMP CHANNEL CONFIGURATION**

Channels[\[GDS GUSBAMP CHANNELS MAX\]](#)

The configuration of each channel. The element at zero-based index i holds the configuration for the channel with one-based channel number $i + 1$.

2.2.4 FUNCTIONS

GDS_GUSBAMP_GetDeviceInformation

Retrieves extended information about the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_GetDeviceInformation([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], char (*deviceInfo)[[GDS GUSBAMP DEVICE INFORMATION LENGTH MAX](#)])

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	A pointer to the name of the device to set the scaling values for.
deviceInformation	A pointer to a fixed size array of a number of GDS GUSBAMP DEVICE INFORMATION LENGTH MAX characters that receives the device information string.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>deviceInformation</i> is NULL .

GDS_GUSBAMP_GetSupportedSamplingRates

Gets a list of supported sampling rates and their supported features.

[GDS_RESULT](#) GDS_GUSBAMP_GetSupportedSamplingRates([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GUSBAMP_SAMPLING_RATE_FEATURES](#) *supportedSamplingRates, size_t *supportedSamplingRatesCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of supported sampling rates for.
deviceName	A pointer to the name of the device to get the list of supported sampling rates for.
supportedSamplingRates	A pointer to the first element of an allocated array of type GDS_GUSBAMP_SAMPLING_RATE_FEATURES and size <i>supportedSamplingRatesCount</i> that receives the list of supported sampling rates, or NULL to receive the number of supported sampling Rates in <i>supportedSamplingRatesCount</i> only.
supportedSamplingRatesCount	A pointer to a variable that receives the number of supported sampling rates for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedSamplingRatesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedSamplingRates</i> array specified in <i>supportedSamplingRatesCount</i> is too small to retrieve the total list of supported sampling rates and <i>supportedSamplingRates</i> is not NULL .

Remarks

supportedSamplingRates is a pointer to the first element of an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call [GDS GUSBAMP GetSupportedSamplingRates](#) with *supportedSamplingRates* set to [NULL](#). The function then returns the number of elements of the *supportedSamplingRates* array in *supportedSamplingRatesCount*. Allocate a [GDS GUSBAMP SAMPLING RATE FEATURES](#) array of that size and pass it on to another call of [GDS GUSBAMP GetSupportedSamplingRates](#) to retrieve the list of supported sampling rates.

GDS_GUSBAMP_GetBandpassFilters

Gets the list of available bandpass filters for the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GUSBAMP_GetBandpassFilters([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS_FILTER_INFO](#) *bandpassFilters, size_t *bandpassFiltersCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available bandpass filters for.
deviceName	A pointer to the name of the device to get the list of available bandpass filters for.
bandpassFilters	A pointer to the first element of an allocated array of type GDS_FILTER_INFO and size <i>bandpassFiltersCount</i> , or NULL to receive the number of available bandpass filters in <i>bandpassFiltersCount</i> only.
bandpassFiltersCount	A pointer to a variable that receives the number of available bandpass filters for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>bandpassFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>bandpassFilters</i> array specified in <i>bandpassFiltersCount</i> is too small to retrieve the total list of available bandpass filters and <i>bandpassFilters</i> is not NULL .

Remarks

bandpassFilters is a pointer to an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call [GDS_GUSBAMP_GetBandpassFilters](#) with *bandpassFilters* set to [NULL](#). The function then returns the number of elements of the

bandpassFilters array in *bandpassFiltersCount*. Allocate a [GDS_FILTER_INFO](#) array of that size and pass it on to another call of [GDS_GUSBAMP_GetBandpassFilters](#) to retrieve the list of available bandpass filters.

GDS_GUSBAMP_GetNotchFilters

Gets the list of available notch filters for the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_GetNotchFilters([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS FILTER INFO](#) *notchFilters, size_t *notchFiltersCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available notch filters for.
deviceName	A pointer to the name of the device to get the list of available notch filters for.
notchFilters	A pointer to the first element of an allocated array of type GDS FILTER INFO and size <i>notchFiltersCount</i> , or NULL to receive the number of available notch filters in <i>notchFiltersCount</i> only.
notchFiltersCount	A pointer to a variable that receives the number of available notch filters for the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>notchFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>notchFilters</i> array specified in <i>notchFiltersCount</i> is too small to retrieve the total list of available notch filters and <i>notchFilters</i> is not NULL .

Remarks

notchFilters is a pointer to an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call [GDS GUSBAMP_GetNotchFilters](#) with *notchFilters*

set to [NULL](#). The function then returns the number of elements of the *notchFilters* array in *notchFiltersCount*. Allocate a [GDS FILTER INFO](#) array of that size and pass it on to another call of [GDS GUSBAMP GetNotchFilters](#) to retrieve the list of available notch filters.

GDS_GUSBAMP_GetAsyncDigitalIOs

Gets the current values of all asynchronous digital input and output channels for the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GUSBAMP_GetAsyncDigitalIOs([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNEL](#) (*asyncDigitalIOs)[[GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNELS_MAX](#)])

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to get the current asynchronous digital I/O channels's current values for.
deviceName	A pointer to the name of the device to get the asynchronous digital I/O channels's current values for.
asyncDigitalIOs	A pointer to a fixed size array of a number of exactly GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNELS_MAX elements of type GDS_GUSBAMP_ASYNC_DIGITAL_IO_CHANNEL that receives the current values of all provided asynchronous digital input and output channels.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>asyncDigitalIOs</i> is NULL .

GDS_GUSBAMP_SetAsyncDigitalOutputs

Sets or changes the values of the specified asynchronous digital output channels for the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_SetAsyncDigitalOutputs([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GUSBAMP ASYNC DIGITAL IO CHANNEL](#) *asyncDigitalOutputs, size_t asyncDigitalOutputsCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device whose asynchronous digital output channel values should be set or changed.
deviceName	A pointer to the name of the device whose asynchronous digital output channel values should be set or changed.
asyncDigitalOutputs	Contains the channels to modify together with their new values.
asyncDigitalOutputsCount	The number of elements in the specified <i>asyncDigitalOutputs</i> array.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or at least one element in <i>asyncDigitalOutputs</i> specifies an input channel in its GDS GUSBAMP ASYNC DIGITAL IO CHANNEL::Direction field.
GDS_ERROR_UNKNOWN	At least one element in <i>asyncDigitalOutputs</i> specifies an invalid channel number in its GDS GUSBAMP ASYNC DIGITAL IO CHANNEL::Channel Number field.

Remarks

Only the output channels that are specified in the *asyncDigitalOutputs* array are modified; others preserve their current state.

g.USBamp version 3.0 devices accept channel numbers from 1 to 4.
g.USBamp version 2.0 devices accept channel numbers from 1 to 2.

GDS_GUSBAMP_GetScaling

Gets the scaling values for the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_GetScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GUSBAMP SCALING](#) *scaling)

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to get the scaling values for.
deviceName	A pointer to the name of the device to get the scaling values for.
scaling	A pointer to an allocated GDS GUSBAMP SCALING structure that receives the scaling values for the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running.

GDS_GUSBAMP_SetScaling

Sets the scaling values for the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GUSBAMP_SetScaling([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GUSBAMP_SCALING](#) *scaling)

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	A pointer to the name of the device to set the scaling values for.
scaling	A pointer to an initialized GDS_GUSBAMP_SCALING structure that contains the scaling values to set.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running.

GDS_GUSBAMP_Calibrate

Calculates new scaling values for each channel under the current conditions of the specified g.USBamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_Calibrate([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GUSBAMP SCALING](#) *scaling)

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to calibrate.
deviceName	A pointer to the name of the device to calibrate.
scaling	A pointer to an allocated GDS GUSBAMP SCALING structure that receives the calculated scaling values for the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or an error occurred during calibration.

Remarks

The calculated scaling values are not applied on the device yet. Use [GDS_GUSBAMP_SetScaling](#) with the returned *scaling* structure to apply them on the device.

GDS_GUSBAMP_GetImpedance

Measures the impedance at the specified channel of the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

This method is deprecated (use [GDS_GUSBAMP_GetImpedanceEx](#) instead).

[GDS_RESULT](#) GDS_GUSBAMP_GetImpedance([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], unsigned char channelNumber, double *impedance)

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to use for impedance measurement.
deviceName	A pointer to the name of the device to use for impedance measurement.
channelNumber	The one-based number of the channel whose impedance should be measured.
impedance	A pointer to a variable that receives the impedance value of the specified channel in ohm.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>impedance</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or <i>channelNumber</i> specifies an invalid channel number.

Remarks

All grounds are connected to common ground during impedance measurement. Impedance is measured between the specified channel and ground.

Impedance can be measured on ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for *channelNumber*:

Value for channel	Description
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

GDS_GUSBAMP_GetImpedanceEx

Measures the impedance at the specified channel of the specified g.USBamp device of the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GUSBAMP_GetImpedanceEx([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], unsigned char channelNumber, double *impedance)

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to use for impedance measurement.
deviceName	A pointer to the name of the device to use for impedance measurement.
channelNumber	The one-based number of the channel whose impedance should be measured.
impedance	A pointer to a variable that receives the impedance value of the specified channel in ohm.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>impedance</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or <i>channelNumber</i> specifies an invalid channel number.

Remarks

All grounds are connected to common ground during impedance measurement. Impedance is measured between the specified channel and ground.

Impedance can be measured on ordinary synchronous analog input channels and each group's reference channel. The following table lists valid values for *channelNumber*:

Value for channel	Description
1 to 16	Synchronously sampled analog input channels 1 to 16.
17	Reference channel of group A (REFA).
18	Reference channel of group B (REFB).
19	Reference channel of group C (REFC).
20	Reference channel of group D (REFD).

All returned impedance values are rounded to the nearest integral ohm value. Negative values represent different error codes.

The following table lists the recommended classification of returned impedances and error codes for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

Impedances for active electrodes connected via a g.GAMMAbox cannot be measured with g.USBamp devices.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The impedance could not be calculated but is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The impedance could not be calculated but is estimated to be very high.

2.3 G.HIAMP SPECIFICS (GDSCLIENTAPI_GHIAMP.H)

The methods and symbols described in this section only apply for g.HIamp devices.

2.3.1 CONSTANTS

GDS_GHIAMP_CHANNELS_MAX

The maximum number of channels a g.HIamp device provides.

GDS_GHIAMP_DEVICE_INFORMATION_LENGTH_MAX

The maximum number of characters the device info string can contain, including the terminating null character.

2.3.2 STRUCTURES

GDS_GHIAMP_SAMPLING_RATE_FEATURES

The supported features at a specific sampling rate.

Public Fields

- `uint32_t` **SamplingRate**
The sampling rate that the features of the current instance belong to.
- `uint16_t` **RecommendedNumberOfScans**
The recommended number of scans for *SamplingRate*.

GDS_GHIAMP_SCALING

The scaling values of each channel of a single g.HIamp device.

Public Fields

- `float` **Offset**[\[GDS_GHIAMP_CHANNELS_MAX\]](#)
The offset for each channel in μV .
- `float` **Factor**[\[GDS_GHIAMP_CHANNELS_MAX\]](#)
The scaling factor for each channel.

GDS_GHIAMP_SIGNAL_GENERATOR_CONFIGURATION

The configuration of the internal signal generator.

Public Fields

- `BOOL` **Enabled**
Indicates whether the internal signal generator should be enabled.
- `uint16_t` **Frequency**
The frequency of the signal generator in hertz. Valid values range from 1 Hz to 700 Hz.

GDS_GHIAMP_CHANNEL_CONFIGURATION

The configuration of a single channel.

Public Fields

- `BOOL` **Acquire**
Indicates whether the channel that this configuration belongs to should be included in data acquisition.

- **int32_t BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GDS_GHIAMP_GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- **int32_t NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GDS_GHIAMP_GetNotchFilters](#). A value of -1 indicates that no notch filter is used for the channel.
- **uint16_t ReferenceChannel**
The one-based number of the channel to use as a reference for the current channel, or zero if no reference should be used for the current channel.

GDS_GHIAMP_CONFIGURATION

The configuration of a g.Hlamp device.

Public Fields

- **uint32_t SamplingRate**
The sampling rate of the synchronously sampled inputs in hertz. Use [GDS_GHIAMP_GetSupportedSamplingRates](#) for a list of valid values.
- **uint16_t NumberOfScans**
The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting *NumberOfScans* to a value other than zero is recommended for advanced users only. This value depends on the value of *SamplingRate*. The following table lists the recommended values of *NumberOfScans* for all possible sample rates.

Sampling Rate [Hz]	Recommended value for NumberOfScans
256	8
512	16
600	32
1200	64
2400	128
4800	256
9600	256
19200	256
38400	256

- **BOOL TriggerLinesEnabled**
Indicates whether the synchronously sampled digital trigger input channels should be included in data acquisition. If enabled, the values of the digital trigger input channels come encoded in a single value as an additional channel appended to the analog channels.
For g.Hlamp, there are 16 trigger lines encoded in this value. The eight least significant

bits (bits 1 to 8) contain the digital values (0 for LOW or 1 for HIGH) of trigger channels 1 to 8. Bits 17 to 24 contain the digital values of trigger channels 9 to 16. Bits 9 to 16 and bits 25 to 32 of the value are not used.

- **[BOOL](#) HoldEnabled**
Indicates whether the signal hold input is enabled. If the signal hold input is enabled, a HIGH level on the HOLD input causes the amplifier to latch the input channels' current values, and ignore the actually applied signal until the HOLD input is pulled down to LOW again. If the signal hold input is disabled, the signal applied to the HOLD input has no effect.
- **[BOOL](#) CounterEnabled**
Indicates whether a sample counter should be applied on the very first channel selected for acquisition. If enabled, a sample counter is applied to the very first channel that is selected for acquisition. The actually applied signal on that channel is then disregarded. The counter overruns at 1,000,000 samples.
- **[GDS_GHIAMP_SIGNAL_GENERATOR_CONFIGURATION](#) InternalSignalGenerator**
The configuration of the internal signal generator.
- **[GDS_GHIAMP_CHANNEL_CONFIGURATION](#) Channels**[\[GDS_GHIAMP_CHANNELS_MAX\]](#)
The configuration of each channel. An element at zero-based index i holds the configuration of the channel with one-based channel number $i + 1$. Channels that the device does not provide cannot be selected for acquisition. The other settings in their configuration structure will be ignored.

2.3.3 FUNCTIONS

GDS_GHIAMP_GetDeviceInformation

Retrieves extended information about the specified g.HIamp device of the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GHIAMP_GetDeviceInformation([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], char (*deviceInformation)[[GDS_GHIAMP_DEVICE_INFORMATION_LENGTH_MAX](#)])

Parameters	
connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	A pointer to the name of the device to set the scaling values for.
deviceInformation	A pointer to a fixed size array of a number of GDS_GHIAMP_DEVICE_INFORMATION_LENGTH_MAX characters that receives the device information string.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL or <i>deviceInformation</i> is NULL .

GDS_GHIAMP_GetAvailableChannels

Gets a Boolean flag for each possible channel a device in its full configuration may provide, indicating whether the corresponding channel is actually provided and available on the device.

[GDS_RESULT](#) GDS_GHIAMP_GetAvailableChannels([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [BOOL](#) (*availableChannels)[[GDS_GHIAMP_CHANNELS_MAX](#)])

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available channels for.
deviceName	A pointer to the name of the device to get the list of available channels for.
availableChannels	A pointer to an allocated array of length GDS_GHIAMP_CHANNELS_MAX where an element at index i receives a boolean flag indicating whether the corresponding channel with one-based channel number $i + 1$ is available on the device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>availableChannels</i> is NULL .

GDS_GHIAMP_GetSupportedSamplingRates

Gets a list of supported sampling rates and their supported features.

[GDS_RESULT](#) GDS_GHIAMP_GetSupportedSamplingRates([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GHIAMP_SAMPLING_RATE_FEATURES](#) *supportedSamplingRates, size_t *supportedSamplingRatesCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of supported sampling rates for.
deviceName	A pointer to the name of the device to get the list of supported sampling rates for.
supportedSamplingRates	A pointer to the first element of an allocated array of type GDS_GHIAMP_SAMPLING_RATE_FEATURES and size <i>supportedSamplingRatesCount</i> that receives the list of supported sampling rates, or NULL to receive the number of supported sampling Rates in <i>supportedSamplingRatesCount</i> only.
supportedSamplingRatesCount	A pointer to a variable that receives the number of supported sampling rates for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedSamplingRatesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedSamplingRates</i> array specified in <i>supportedSamplingRatesCount</i> is too small to retrieve the total list of supported sampling rates and <i>supportedSamplingRates</i> is not NULL .

Remarks

supportedSamplingRates is a pointer to the first element of an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call [GDS GHIAMP GetSupportedSamplingRates](#) with *supportedSamplingRates* set to [NULL](#). The function then returns the number of elements of the *supportedSamplingRates* array in *supportedSamplingRatesCount*. Allocate a [GDS GHIAMP SAMPLING RATE FEATURES](#) array of that size, then pass it on to another call of [GDS GHIAMP GetSupportedSamplingRates](#) to retrieve the list of supported sampling rates.

GDS_GHIAMP_GetBandpassFilters

Gets the list of available bandpass filters for the specified g.HIamp device for the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GHIAMP_GetBandpassFilters([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_FILTER_INFO](#) *bandpassFilters, size_t *bandpassFiltersCount)

Parameters

connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available bandpass filters for.
deviceName	A pointer to the name of the device to get the list of available bandpass filters for.
bandpassFilters	A pointer to the first element of an allocated array of type GDS_FILTER_INFO and size <i>bandpassFiltersCount</i> , or NULL to receive the number of available bandpass filters in <i>bandpassFiltersCount</i> only.
bandpassFiltersCount	A pointer to a variable that receives the number of available bandpass filters for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>bandpassFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>bandpassFilters</i> array specified in <i>bandpassFiltersCount</i> is too small to retrieve the total list of available bandpass filters and <i>bandpassFilters</i> is not NULL .

Remarks

bandpassFilters is a pointer to the first element of an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call

[GDS GHIAMP GetBandpassFilters](#) with *bandpassFilters* set to [NULL](#). The function then returns the number of elements of the *bandpassFilters* array in *bandpassFiltersCount*. Allocate a [GDS FILTER INFO](#) array of that size, then pass it on to another call of [GDS GHIAMP GetBandpassFilters](#) to retrieve the list of available bandpass filters.

GDS_GHIAMP_GetNotchFilters

Gets the list of available notch filters for the specified g.HIamp device for the data acquisition session associated with the specified connection handle.

[GDS_RESULT](#) GDS_GHIAMP_GetNotchFilters([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_FILTER_INFO](#) *notchFilters, size_t *notchFiltersCount)

Parameters	
connectionHandle	The connection handle whose associated data acquisition contains the device to get the list of available notch filters for.
deviceName	A pointer to the name of the device to get the list of available notch filters for.
notchFilters	A pointer to the first element of an allocated array of type GDS_FILTER_INFO and size <i>notchFiltersCount</i> , or NULL to receive the number of available notch filters in <i>notchFiltersCount</i> only.
notchFiltersCount	A pointer to a variable that receives the number of available notch filters for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>notchFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>notchFilters</i> array specified in <i>notchFiltersCount</i> is too small to retrieve the total list of available notch filters and <i>notchFilters</i> is not NULL .

Remarks

notchFilters is a pointer to the first element of an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call [GDS_GHIAMP_GetNotchFilters](#) with *notchFilters* set to [NULL](#). The function then returns the number

of elements of the *notchFilters* array in *notchFiltersCount*. Allocate a [GDS_FILTER_INFO](#) array of that size, then pass it on to another call of [GDS_GHIAMP_GetNotchFilters](#) to retrieve the list of available notch filters.

GDS_GHIAMP_GetFactoryScaling

Gets the scaling values initially set by the manufacturer for the specified g.HIamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GHIAMP_GetFactoryScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GHIAMP SCALING](#) *factoryScaling)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to get the factory scaling values for.
<i>deviceName</i>	A pointer to the name of the device to get the factory scaling values for.
<i>factoryScaling</i>	A pointer to an allocated GDS GHIAMP SCALING structure that receives the factory scaling values of the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>factoryScaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running.

GDS_GHIAMP_GetScaling

Gets the scaling values for the specified g.HIamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GHIAMP_GetScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GHIAMP SCALING](#) *scaling)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to get the scaling values for.
<i>deviceName</i>	A pointer to the name of the device to get the scaling values for.
<i>scaling</i>	A pointer to an allocated GDS GHIAMP SCALING structure that receives the scaling values of the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running.

GDS_GHIAMP_SetScaling

Sets the scaling values for the specified g.HIamp device for the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GHIAMP_SetScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GHIAMP SCALING](#) *scaling)

Parameters

connectionHandle	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
deviceName	A pointer to the name of the device to set the scaling values for.
scaling	A pointer to an initialized GDS GHIAMP SCALING structure that contains the scaling values to set.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running.

Remarks

[GDS_GHIAMP_Calibrate](#) may return NaN values for channels it cannot calculate scaling values for. Any NaN values in *scaling*'s [GDS_GHIAMP_SCALING::Factor](#) array will be replaced by 1 before they're sent to the device. Any NaN values in *scaling*'s [GDS_GHIAMP_SCALING::Offset](#) array will be replaced by 0 before they're sent to the device.

GDS_GHIAMP_Calibrate

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions, if the calibration set is applied.

[GDS_RESULT](#) GDS_GHIAMP_Calibrate([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GHIAMP_SCALING](#) *scaling)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to calibrate.
<i>deviceName</i>	A pointer to the name of the device to calibrate.
<i>scaling</i>	A pointer to an allocated GDS_GHIAMP_SCALING structure that receives the new scaling values calculated under the current conditions for each synchronously sampled analog input channel.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or an error occurred during calibration.

Remarks

All channels must be shorted to ground (GND). This is done by applying the calibration equipment, consisting of four connectors, one for each group (A, B, C, and D). They connect all channel inputs of a group with ground (GND).

The returned values will not be applied unless they're explicitly set with [GDS_GHIAMP_SetScaling](#).

Calibration is required if amplitude and/or offset of the known input signal differs from the measured signal.

The function returns quiet NaN values (`std::numeric_limits<float>::quiet_NaN()`) as the scaling factor and offset for channels that failed calibration.

GDS_GHIAMP_GetImpedance

Measures impedance of all available synchronously sampled analog input channels for active or passive electrodes in ohm.

This method is deprecated (use [GDS_GHIAMP_GetImpedanceEx](#) instead).

[GDS_RESULT](#) GDS_GHIAMP_GetImpedance([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [BOOL](#) (*isActiveElectrode)[[GDS_GHIAMP_CHANNELS_MAX](#)], double (*impedances)[[GDS_GHIAMP_CHANNELS_MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to measure impedance for.
<i>deviceName</i>	A pointer to the name of the device to measure impedance for.
<i>isActiveElectrode</i>	A pointer to an initialized array where an element at zero-based index <i>i</i> indicates whether the electrode connected to one-based channel number <i>i + 1</i> is active (TRUE) or passive (FALSE) (see remarks section of the method for details). The caller must pass this array to the method, since the device cannot determine if active or passive electrodes are connected.
<i>impedances</i>	A pointer to an allocated array that receives the impedance values in ohm for each channel. An element at zero-based index <i>i</i> receives the impedance value in ohm of one-based channel number <i>i + 1</i> , or <i>std::numeric_limits<double>::quiet_NaN()</i> , or an error code (see remarks section of the method for details).

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

<u>GDS_ERROR_SUCCESS</u>	The operation succeeded.
<u>GDS_ERROR_INVALID_HANDLE</u>	<i>connectionHandle</i> is invalid.
<u>GDS_ERROR_HANDLE_NOT_ASSOCIATED</u>	<i>connectionHandle</i> is not associated with any data acquisition session already opened with <u>Connect</u> .
<u>GDS_ERROR_INVALID_DEVICE</u>	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
<u>GDS_ERROR_INVALID_PARAMETER</u>	<i>deviceName</i> is <u>NULL</u> , or <i>isActiveElectrode</i> is <u>NULL</u> , or <i>impedances</i> is <u>NULL</u> .
<u>GDS_ERROR_UNKNOWN</u>	Data acquisition is currently running, or mixed electrode types are specified in the <i>isActiveElectrode</i> array for any of the available channels, or an error occurred during impedance measurement.

Remarks

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used. Active and passive electrodes cannot be mixed. The method will fail if mixed electrode types are specified.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will always receive *NaN* as impedance value. The method will not fail if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

All returned impedance values are rounded to the nearest integral kilo-ohm value. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. Values below 5,000 ohm or above 500,000 ohm represent different error codes and do not rate the severeness of error. Thus, they are not comparable to each other. The method returns quiet *NaN* values (*std::numeric_limits<double>::quiet_NaN()*) for channel 1, for unavailable channels and for channels that did not yield a finite impedance value. The availability of a channel can be determined using [GDS_GHIAMP_GetAvailableChannels](#).

If a TMS box is connected, the caller must subtract 10 kilo-ohm from valid impedance values to get the correct value (do not do this for error codes).

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 5,000	Not measurable, estimated good impedance.
5,000...500,000	Measured impedance values.
> 500,000	Not measurable impedance due to an erroneous signal (maybe due to a not connected electrode, a too big offset, etc.).
std::numeric_limits<double>::quiet_NaN()	For channel 1 and unavailable channels and for channels that yielded non-finite impedance values (such as <i>NaN</i> or infinity).

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
3,000	The impedance could not be calculated but is estimated to be very low.
4,000	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
501,000	The amplitude of the measured signal is too small.
502,000	The offset of the measured signal is too large.
503,000	The RMS value of the measured signal is too large.
504,000	The low frequency power of the measured signal is too large.
505,000	The amplitude of the measured signal at channel 1 is too small.
506,000	The offset, RMS, or low frequency power at channel 1 is too large.

GDS_GHIAMP_GetImpedanceEx

Measures impedance of all available synchronously sampled analog input channels for active or passive electrodes in ohm.

[GDS RESULT](#) GDS_GHIAMP_GetImpedanceEx([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS ELECTRODE TYPE](#) electrodeType, [BOOL](#) tmsBoxCorrection, [BOOL](#) firstMeasurementOfSession, double (*impedances)[[GDS_GHIAMP_CHANNELS MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to measure impedance for.
<i>deviceName</i>	A pointer to the name of the device to measure impedance for.
<i>electrodeType</i>	The type of connected electrodes. All electrodes must be of the same selected type.
<i>tmsBoxCorrection</i>	TRUE if a TMS electrode connector box is connected and impedance values of all channels should be corrected by 10 kilo-ohm; otherwise, FALSE .
<i>firstMeasurementOfSession</i>	If this is the start of a new measurement session (i.e. the first in a series of consecutive calls to GDS_GHIAMP_GetImpedanceEx), set this flag to TRUE to clear internal interim values of previous measurements; for the following repeated calls, set this flag to FALSE until the end of the measurement session.
<i>impedances</i>	A pointer to an allocated array that receives the impedance values in ohm for each channel. An element at zero-based index <i>i</i> receives the impedance value in ohm of one-based channel number <i>i</i> + 1, or <i>std::numeric_limits<double>::quiet_NaN()</i> , or an error code (see remarks section of the method for details).

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

<u>GDS_ERROR_SUCCESS</u>	The operation succeeded.
<u>GDS_ERROR_INVALID_HANDLE</u>	<i>connectionHandle</i> is invalid.
<u>GDS_ERROR_HANDLE_NOT_ASSOCIATED</u>	<i>connectionHandle</i> is not associated with any data acquisition session already opened with <u>Connect</u> .
<u>GDS_ERROR_INVALID_DEVICE</u>	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
<u>GDS_ERROR_INVALID_PARAMETER</u>	<i>deviceName</i> is <u>NULL</u> , or <i>isActiveElectrode</i> is <u>NULL</u> , or <i>impedances</i> is <u>NULL</u> .
<u>GDS_ERROR_UNKNOWN</u>	Data acquisition is currently running, or an error occurred during impedance measurement.

Remarks

Channel 1 and the ground electrode must always be connected for impedance measurement. A special z-electrode must be used on channel 1 if active electrodes are used.

Active and passive electrodes cannot be mixed. The method will fail if mixed electrode types are specified.

Channel 1 plays a special role in impedance measurement and must always be connected. However, its impedance cannot be determined. Channel 1 will always receive *NaN* as impedance value. The method will not fail if an invalid reference signal is measured on channel 1 (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

If a TMS electrode connector box is connected, set the *tmsBoxCorrection* parameter to [TRUE](#): 10 kilo-ohm will be subtracted of all measured impedance values in this case due to internal dropping resistors. Hence, no other type of electrode connector boxes can be connected to the same device if an TMS electrode connector box is connected for impedance measurement.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The method returns quiet *NaN* values (*std::numeric_limits<double>::quiet_NaN()*) for channel 1, for unavailable channels and for channels that did not yield a finite impedance value. The availability of a channel can be determined using [GDS_GHIAMP_GetAvailableChannels](#).

The following table lists the recommended classification of returned impedances for passive gel electrodes:

Impedance value in ohm	Recommended classification for passive gel electrodes
< 7,500	Good impedance.
7,500...20,000	Medium impedance.
> 20,000	Bad impedance.

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too big.
-5	The RMS value of the measured signal is too big.
-6	The drift of the measured signal is too big.
-9	The amplitude of the measured signal at channel 1 is too small.
-10	The offset, RMS, or drift at channel 1 is too big.
std::numeric_limits<double>::quiet_NaN()	For channel 1 and for unavailable channels and for channels that yielded non-finite impedance values (such as <i>NaN</i> or infinity).

2.4 G.NAUTILUS SPECIFICS (GDSCLIENTAPI_GNAUTILUS.H)

The methods and symbols described in this section only apply for g.Nautilus devices.

2.4.1 CONSTANTS

GDS_GNAUTILUS_MODULES_MAX

The maximum number of modules that can be mounted on a g.Nautilus device.

GDS_GNAUTILUS_CHANNELS_PER_MODULE_MAX

The maximum number of channels within a single module.

GDS_GNAUTILUS_CHANNELS_MAX

The maximum number of electrode channels a g.Nautilus device provides.

GDS_GNAUTILUS_ADDITIONAL_CHANNELS_MAX

The maximum number of channels a g.Nautilus device provides in addition to the electrode channels (e.g.: counter, acceleration data, ...).

GDS_GNAUTILUS_DIGITAL_IO_CHANNELS_MAX

The number of asynchronous digital IN channels a g.Nautilus device provides.

GDS_GNAUTILUS_ELECTRODE_NAME_LENGTH_MAX

The number of the maximum length of an electrode name.

GDS_GNAUTILUS_DEVICE_INFORMATION_LENGTH_MAX

The maximum length of the device information string.

2.4.2 ENUMERATIONS

GDS_GNAUTILUS_INPUT_SIGNAL

The input signals provided for a channel of a g.Nautilus device.

GDS_GNAUTILUS_INPUT_SIGNAL_ELECTRODE	Analog input from electrodes.
GDS_GNAUTILUS_INPUT_SIGNAL_SHORTED	Channels inputs shorted to GND.
GDS_GNAUTILUS_INPUT_SIGNAL_TEST_SIGNAL	Test Signal as input for all electrodes.

2.4.3 FUNCTION POINTERS

GDS_GNAUTILUS_ImpedanceMeasurementCallback

The type of functions that handle callbacks of completed asynchronous impedance measurement cycles.

```
typedef void (*GDS_GNAUTILUS_ImpedanceMeasurementCallback)(GDS\_HANDLE
connectionHandle, char (*deviceName)[DEVICE\_NAME\_LENGTH\_MAX], double
(*impedances)[GDS\_GNAUTILUS\_CHANNELS\_MAX], BOOL aborted, GDS\_RESULT result, void*
userData)
```

Parameters

<i>connectionHandle</i>	The handle associated with the session that triggered the callback.
<i>deviceName</i>	A pointer to the name of the device that triggered the callback.
<i>impedances</i>	A pointer to an allocated array that receives the impedance values in ohm for each channel. An element at zero-based index <i>i</i> receives the impedance value in ohm of zero-based channel number <i>i</i> , or <i>std::numeric_limits<double>::quiet_NaN()</i> (see extended description of GDS_GNAUTILUS_GetImpedanceEx / GDS_GNAUTILUS_GetImpedance for details).
<i>aborted</i>	Indicates whether an error that occurred is crucial for impedance measurement, such that measurement had to be stopped automatically.
<i>userData</i>	Can be used to pass data from the user to the callback.

2.4.4 STRUCTURES

GDS_GNAUTILUS_DIGITAL_IO_CHANNEL

Describes the state of an asynchronous digital IN channel.

Public Fields

- unsigned int **ChannelNumber**
The one-based channel number of the digital IN channel described by this instance.
- [GDS_CHANNEL_DIRECTION](#) **Direction**
Indicates if the channel described by this instance is a digital input or output channel.

GDS_GNAUTILUS_SCALING

The scaling values for each channel of a single g.Nautilus device.

Public Fields

- float **Offset**[\[GDS_GNAUTILUS_CHANNELS_MAX\]](#)
The offset value for each channel in μV .
- float **Factor**[\[GDS_GNAUTILUS_CHANNELS_MAX\]](#)
The scaling factor for each channel.

GDS_GNAUTILUS_CHANNEL_CONFIGURATION

- Describes the configuration of a single channel of a g.Nautilus device.

Public Fields

- [BOOL](#) **Enabled**
Indicates whether the channel is enabled for data acquisition or not.
- double **Sensitivity**
The input sensitivity of the channel in microvolts.
- [BOOL](#) **UsedForNoiseReduction**
Indicates whether the channel is used for noise reduction computation or not.
- [BOOL](#) **UsedForCar**
Indicates whether the channel is used for common average reference (CAR) calculation.
- int32_t **BandpassFilterIndex**
The zero-based index of the bandpass filter to use for the channel. The index is related to the list of available bandpass filters that are returned by [GDS_GNAUTILUS_GetBandpassFilters](#). A value of -1 indicates that no bandpass filter is used for the channel.
- int32_t **NotchFilterIndex**
The zero-based index of the notch filter to use for the channel. The index is related to the list of available notch filters that are returned by [GDS_GNAUTILUS_GetNotchFilters](#). A value of -1 indicates that no notch filter is used for the channel.
- int32_t **BipolarChannel**
The bipolar derivation setting for the channel. Set this value to -1 if unipolar derivation should be used with ground and reference electrodes. Otherwise, if bipolar derivation should be performed for the channel that this configuration belongs to, set *BipolarChannel* to the zero-based channel number whose signal should be subtracted from the measured signal of this channel.

GDS_GNAUTILUS_CONFIGURATION

Describes the configuration of a g.Nautilus device.

Public Fields

- [BOOL](#) **Slave**
Indicates whether the g.Nautilus device is operated in slave mode or not.
- [GDS_GNAUTILUS_INPUT_SIGNAL](#) **InputSignal**
Specifies the input signal of all channels for the device.
- `uint32_t` **SamplingRate**
The sampling rate of the g.Nautilus device in Hertz. Use [GDS_GNAUTILUS_GetSupportedSamplingRates](#) for a list of valid values.
- `uint32_t` **NumberOfScans**
The number of scans to retrieve from the device at once, or zero to apply an automatically determined number of scans for the specified sampling rate. Setting *NumberOfScans* to a value other than zero is recommended for advanced users only. This value depends on the value of *SamplingRate*. The following table lists the recommended values of *NumberOfScans* for all possible sampling rates.

Sampling Rate [Hz]	Recommended value for <i>NumberOfScans</i>
250	8
512	15

- [BOOL](#) **NoiseReduction**
Indicates whether noise reduction should be enabled.
- [BOOL](#) **CAR**
Indicates whether common average reference (CAR) calculation should be enabled.
- [BOOL](#) **AccelerationData**
Indicates whether acceleration data channels should be included in data acquisition. If enabled, the acceleration values of the x, y, and z axis come as three additional channels appended to the analog channels. Acceleration values range within ± 6 g.
- [BOOL](#) **Counter**
Indicates whether the counter channel should be included in data acquisition. If enabled, counter comes as an additional channel appended to the acceleration data channels. The counter value is the number of the current sample.
- [BOOL](#) **LinkQualityInformation**
Indicates whether the link quality information channel should be included in data acquisition. If enabled, link quality information comes as an additional channel appended to the counter channel. Link quality information values range within 0 and 100 percent.
- [BOOL](#) **BatteryLevel**
Indicates whether the battery level (state of charge) channel should be included in data acquisition. If enabled, battery level comes as an additional channel appended to the link quality information channel. Battery level values range within 0 and 100 percent.
- [BOOL](#) **DigitalIOs**
Indicates whether the synchronously sampled digital input channels should be included in data acquisition. If enabled, the values of the digital trigger input channels come bit-encoded in a single additional channel appended to the battery level channel.
- [BOOL](#) **ValidationIndicator**
Indicates whether the validation indicator channel should be included in data acquisition.

If enabled, the value of the validation indicator comes as an additional channel appended to the digital IO channel. Valid validation indicator values are 0 and 1.

- `uint32_t` **NetworkChannel**

The radio channel used for wireless communication between headset and base station of the device.

- [GDS GNAUTILUS CHANNEL CONFIGURATION](#)

Channels[\[GDS GNAUTILUS CHANNELS MAX\]](#)

The configuration of each channel. An element at zero-based index *i* holds the configuration of the channel with zero-based channel number *i*. Channels that the device does not provide cannot be selected for acquisition. The other settings in their configuration structure will be ignored.

2.4.5 FUNCTIONS

GDS_GNAUTILUS_GetDeviceInformation

Retrieves extended information about the specified device.

[GDS_RESULT](#) GDS_GNAUTILUS_GetDeviceInformation([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], char (*deviceInformation)[[GDS_GNAUTILUS_DEVICE_INFORMATION_LENGTH_MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve extended device information for.
<i>deviceName</i>	A pointer to the name of the device to retrieve extended device information for.
<i>deviceInformation</i>	A pointer to the extended device information string.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>deviceInformation</i> is NULL .

GDS_GNAUTILUS_GetChannelNames

Retrieves the number of actually mounted modules of a device and the names of all available channels.

[GDS RESULT](#) GDS_GNAUTILUS_GetChannelNames([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], uint32_t *mountedModulesCount, char (*electrodeNames)[[GDS_GNAUTILUS_ELECTRODE_NAME_LENGTH_MAX](#)], size_t *electrodeNamesCount)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve the device information from.
<i>deviceName</i>	A pointer to the name of the device to retrieve the device information from.
<i>mountedModulesCount</i>	A pointer to the variable that receives the number of mounted modules.
<i>electrodeNames</i>	A pointer to an array that holds the names of the numbered channels as strings (e.g. PO4, C3,...). An element at zero-based index <i>i</i> contains the electrode name of the channel with zero-based channel number <i>i</i> .
<i>electrodeNamesCount</i>	A pointer to the variable that specifies the number of elements in the <i>electrodeNames</i> array.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>mountedModulesCount</i> is NULL , or <i>electrodeNamesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>electrodeNames</i> array specified in <i>electrodeNamesCount</i> is too small to

retrieve the total list of mounted electrodes' names,
and *electrodeNames* is not [NULL](#).

Remarks

The function returns the number of mounted modules within the parameter *mountedModulesCount*, as well as the mapping of the channels within the array *electrodeNames*. The number of modules determines the maximum number of available channels.

To retrieve the number of modules and the length of the array, call the method with *electrodeNames* set to [NULL](#). After allocating memory for the array, call the method again to retrieve the array holding the electrode names.

GDS_GNAUTILUS_GetAvailableChannels

Retrieves a Boolean flag for each possible channel a device in its full configuration may provide. The flag indicates whether the corresponding channel is actually provided and available on the device.

[GDS_RESULT](#) GDS_GNAUTILUS_GetAvailableChannels([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [BOOL](#) (*availableChannels)[[GDS_GNAUTILUS_CHANNELS_MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to get the list of available channels for.
<i>deviceName</i>	A pointer to the name of the device to get the list of available channels for.
<i>availableChannels</i>	A pointer to an allocated array of length GDS_GNAUTILUS_CHANNELS_MAX where an element at index i receives a boolean flag indicating whether the corresponding channel with one-based channel number $i + 1$ is available on the device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>availableChannels</i> is NULL .

GDS_GNAUTILUS_GetAvailableDigitalIOs

Gets information about available digital I/O channels for the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GNAUTILUS_GetAvailableDigitalIOs([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GNAUTILUS DIGITAL IO CHANNEL](#) (*availableDigitalIOChannels)[[GDS GNAUTILUS DIGITAL IO CHANNELS MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to get information about available digital I/O channels for.
<i>deviceName</i>	A pointer to the name of the device to get information about available digital I/O channels for.
<i>availableDigitalIOChannels</i>	A pointer to a fixed size array of a number of exactly GDS GNAUTILUS DIGITAL IO CHANNELS MAX elements of type GDS GNAUTILUS DIGITAL IO CHANNEL that receives information about available digital I/O channels.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>availableDigitalIOChannels</i> is NULL .

GDS_GNAUTILUS_GetSupportedSamplingRates

Gets a list of supported sampling rates.

[GDS_RESULT](#) GDS_GNAUTILUS_GetSupportedSamplingRates([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], uint32_t *supportedSamplingRates, size_t *supportedSamplingRatesCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to get the list of supported sampling rates for.
<i>deviceName</i>	A pointer to the name of the device to get the list of supported sampling rates for.
<i>supportedSamplingRates</i>	A pointer to the first element of an allocated array of size <i>supportedSamplingRatesCount</i> that receives the list of supported sampling rates, or NULL to receive the number of supported sampling rates in <i>supportedSamplingRatesCount</i> only.
<i>supportedSamplingRatesCount</i>	A pointer to a variable that receives the number of supported sampling rates for the specified device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedSamplingRatesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedSamplingRates</i> array specified in <i>supportedSamplingRatesCount</i> is too small to retrieve the total list of supported sampling rates and <i>supportedSamplingRates</i> is not NULL .

Remarks

supportedSamplingRates is a pointer to the first element of an array that must be allocated by the caller already. To determine the number of elements of the array to allocate, call

[GDS GNAUTILUS GetSupportedSamplingRates](#) with *supportedSamplingRates* set to [NULL](#). The function then returns the number of elements of the *supportedSamplingRates* array in *supportedSamplingRatesCount*. Allocate an array of type *uint32_t* of that size, then pass it on to another call of [GDS GNAUTILUS GetSupportedSamplingRates](#) to retrieve the list of supported sampling rates.

GDS_GNAUTILUS_GetSupportedSensitivities

Retrieves the supported sensitivity values.

[GDS_RESULT](#) GDS_GNAUTILUS_GetSupportedSensitivities([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], double *supportedSensitivities, size_t *supportedSensitivitiesCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve the supported sensitivity values for.
<i>deviceName</i>	A pointer to the name of the device to retrieve the supported sensitivity values for.
<i>supportedSensitivities</i>	The array the sensitivity values will be stored in.
<i>supportedSensitivitiesCount</i>	A pointer to the variable that holds the number of available sensitivity values.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedSensitivitiesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedSensitivities</i> array specified in <i>supportedSensitivitiesCount</i> is too small to retrieve the total list of supported sensitivities and <i>supportedSensitivities</i> is not NULL .

Remarks

To retrieve the supported sensitivity values, the method has to be called twice. First, call it with parameter *supportedSensitivities* = [NULL](#), then the method returns the number of available sensitivity values. Using this number, allocate an array of size *supportedSensitivitiesCount* and call the method again to retrieve the supported sensitivities themselves.

GDS_GNAUTILUS_GetSupportedNetworkChannels

Retrieves the supported network channel numbers.

[GDS_RESULT](#) GDS_GNAUTILUS_GetSupportedNetworkChannels([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], uint32_t *supportedNetworkChannels, size_t *supportedNetworkChannelsCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve the supported network channels for.
<i>deviceName</i>	A pointer to the name of the device to retrieve the supported network channels for.
<i>supportedNetworkChannels</i>	The array where the network channel values will be stored in.
<i>supportedNetworkChannelsCount</i>	A pointer to the variable that holds the number of available network channels.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedNetworkChannelsCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedNetworkChannels</i> array specified in <i>supportedNetworkChannelsCount</i> is too small to retrieve the total list of supported sampling rates, and <i>supportedNetworkChannels</i> is not NULL .

Remarks

To retrieve the supported network channel values, the method has to be called twice. First, call it with parameter *supportedNetworkChannels* = [NULL](#), then the method returns the number of available network channels. Using this number, allocate an array of size *supportedNetworkChannelsCount* and call the method again to retrieve the channels themselves.

GDS_GNAUTILUS_GetSupportedInputSources

Retrieves the supported input sources.

[GDS_RESULT](#) GDS_GNAUTILUS_GetSupportedInputSources([GDS_HANDLE](#) connectionHandle, char (*deviceName) [[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GNAUTILUS_INPUT_SIGNAL](#) *supportedInputSources, size_t *supportedInputSourcesCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve the supported input sources for.
<i>deviceName</i>	A pointer to the name of the device to retrieve the supported input sources for.
<i>supportedInputSources</i>	The array where the input sources will be stored in.
<i>supportedInputSourcesCount</i>	A pointer to the variable that holds the number of available input sources.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>supportedInputSourcesCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>supportedInputSources</i> array specified in <i>supportedInputSourcesCount</i> is too small to retrieve the total list of supported sampling rates, and <i>supportedInputSources</i> is not NULL .

Remarks

To retrieve the supported input sources, the method has to be called twice. First, call it with parameter *supportedInputSources* = [NULL](#), then the method returns the number of available input sources. Using this number, allocate an array of size *supportedInputSourcesCount* and call the method again to retrieve the input sources themselves.

GDS_GNAUTILUS_SetNetworkChannel

Sets the radio channel to use for wireless communication between headset and basestation.

[GDS_RESULT](#) GDS_GNAUTILUS_SetNetworkChannel([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], uint32_t networkChannel)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to set the specified channel number.
<i>deviceName</i>	A pointer to the name of the device to retrieve for which the specified channel number shall be set.
<i>networkChannel</i>	The network channel number to use.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL .
GDS_ERROR_UNKNOWN	An invalid network channel was specified.

Remarks

The function sets the specified network channel for the headset as well as for the base station, since both need to connect via the same channel. The network channel number specifies the channel that is used for communication between head set and base station. When switching on a g.Nautilus device, it pairs itself automatically to a base station using the first free channel selected by the base station. A free channel is a channel where no communication is running.

Use [GDS_GNAUTILUS_GetSupportedNetworkChannels](#) to retrieve a list of supported network channels.

GDS_GNAUTILUS_GetNetworkChannel

Retrieves the current radio channel used for communication between headset and base station.

[GDS_RESULT](#) GDS_GNAUTILUS_GetNetworkChannel([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], uint32_t *networkChannel)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to retrieve the network channel from.
<i>deviceName</i>	A pointer to the name of the device to retrieve the network channel for.
<i>networkChannel</i>	A pointer to the variable that receives the channel number.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> is NULL , or <i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>networkChannel</i> is NULL .

Remarks

The function retrieves the current network channel number into *networkChannel*. The network channel number specifies the channel that is used for communication between head set and base station. When switching on a g.Nautilus device, it pairs itself automatically to a base station using the first free channel selected by the base station. A free channel is a channel where no communication is running.

GDS_GNAUTILUS_GetScaling

Gets the scaling values for the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GNAUTILUS_GetScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GNAUTILUS SCALING](#) *scaling)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to get the scaling values for.
<i>deviceName</i>	A pointer to the name of the device to get the scaling values for.
<i>scaling</i>	A pointer to an allocated GDS GNAUTILUS SCALING structure that receives the scaling values of the specified device.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> is NULL , or <i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or impedance measurement is currently running.

GDS_GNAUTILUS_SetScaling

Sets the scaling values for the specified g.Nautilus device of the data acquisition session associated with the specified connection handle.

[GDS RESULT](#) GDS_GNAUTILUS_SetScaling([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GNAUTILUS SCALING](#) *scaling)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to set the scaling values for.
<i>deviceName</i>	A pointer to the name of the device to set the scaling values for.
<i>scaling</i>	A pointer to an initialized GDS GNAUTILUS SCALING structure that contains the scaling values to set.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or impedance measurement is currently running.

Remarks

[GDS GNAUTILUS Calibrate](#) may return NaN values for channels it cannot calculate scaling values for.

Any NaN values in *scaling*'s [GDS GNAUTILUS SCALING::Factor](#) array will be replaced by 1 before they're sent to the device.

Any NaN values in *scaling*'s [GDS GNAUTILUS SCALING::Offset](#) array will be replaced by 0 before they're sent to the device.

GDS_GNAUTILUS_ResetScaling

Resets the device's scaling to a neutral scaling.

[GDS_RESULT](#) GDS_GNAUTILUS_ResetScaling([GDS_HANDLE](#) connectionHandle,
char(*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the specified g.Nautilus device to reset scaling for.
<i>deviceName</i>	A pointer to the name of the device to reset scaling for.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or impedance measurement is currently running.

Remarks

In neutral scaling, all offset values are set to 0 and all scaling factors are set to 1.

GDS_GNAUTILUS_Calibrate

Calculates, but does not yet apply, new scaling values for each synchronously sampled analog input channel under the current conditions if the calibration set is applied.

[GDS RESULT](#) GDS_GNAUTILUS_Calibrate([GDS HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE NAME LENGTH MAX](#)], [GDS GNAUTILUS SCALING](#) *scaling)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to calibrate.
<i>deviceName</i>	A pointer to the name of the device which is to be calibrated.
<i>scaling</i>	A pointer to an allocated GDS GNAUTILUS SCALING structure that receives the new scaling values calculated under the current conditions for each synchronously sampled analog input channel.

Returns

A [GDS RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_ERROR_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>scaling</i> is NULL .
GDS_ERROR_UNKNOWN	Data acquisition is currently running, or impedance measurement is currently running, or an error occurred during calibration.

Remarks

The function calibrates the device and stores the new calibration values in the configuration. The returned values will not be applied unless they're explicitly set with [GDS GNAUTILUS SetScaling](#).

Calibration is required if amplitude and/or offset of the known input signal differs from the measured signal.

The function returns quiet NaN values (*std::numeric_limits<float>::quiet_NaN()*) as the scaling factor and offset for channels that failed calibration.

GDS_GNAUTILUS_GetBandpassFilters

Gets the list of available bandpass filters for the device.

[GDS_RESULT](#) GDS_GNAUTILUS_GetBandpassFilters([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_FILTER_INFO](#) *bandpassFilters, size_t *bandpassFiltersCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to get the available bandpass filters for.
<i>deviceName</i>	A pointer to the name of the device for which the bandpass filter list shall be retrieved.
<i>bandpassFilters</i>	A pointer to an allocated array of type GDS_FILTER_INFO and size <i>bandpassFiltersCount</i> , or NULL to retrieve the number of available bandpass filters only.
<i>bandpassFiltersCount</i>	A pointer to a variable that receives the number of available bandpass filters for the g.Nautilus device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with a data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>bandpassFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>bandpassFilters</i> array specified in <i>bandpassFiltersCount</i> is too small to retrieve the total list of available bandpass filters and <i>bandpassFilters</i> is not NULL .

Remarks

The function retrieves the list of available bandpass filters for the specified g.Nautilus device. *bandpassFilters* is a pointer to an array that has to be allocated by the caller (of size *bandpassFiltersCount*). To determine the *bandpassFiltersCount*, call [GDS_GNAUTILUS_GetBandpassFilters](#) with parameter *bandpassFilters* set to [NULL](#). The function will return the number of available bandpass filters within the parameter *bandpassFiltersCount*. Use

this value to allocate the filter array properly, then call [GDS GNAUTILUS GetBandpassFilters](#) a second time to retrieve the filter list.

GDS_GNAUTILUS_GetNotchFilters

Gets the list of available notch filters for the device.

[GDS_RESULT](#) GDS_GNAUTILUS_GetNotchFilters([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_FILTER_INFO](#) *notchFilters, size_t *notchFiltersCount)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition contains the device to get the available notch filters for.
<i>deviceName</i>	A pointer to the name of the device for which the notch filter list shall be retrieved.
<i>notchFilters</i>	A pointer to an allocated array of type GDS_FILTER_INFO and size <i>notchFiltersCount</i> , or NULL to retrieve the number of available notch filters only.
<i>notchFiltersCount</i>	A pointer to a variable that receives the number of available notch filters for the g.Nautilus device.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>notchFiltersCount</i> is NULL .
GDS_ERROR_BUFFER_TOO_SMALL	The size of the specified <i>notchFilters</i> array specified in <i>notchFiltersCount</i> is too small to retrieve the total list of available notch filters and <i>notchFilters</i> is not NULL .

Remarks

The function retrieves the list of available notch filters for the specified g.Nautilus device. *notchFilters* is a pointer to an array that has to be allocated by the caller (of size *notchFiltersCount*). To determine the *notchFiltersCount*, call [GDS_GNAUTILUS_GetNotchFilters](#) with

parameter *notchFilters* set to [NULL](#). The function will return the number of available notch filters within the parameter *notchFiltersCount*. Use this value to allocate the filter array properly and call [GDS GNAUTILUS GetNotchFilters](#) a second time to retrieve the filter list.

GDS_GNAUTILUS_GetImpedance

Measures impedance of all available synchronously sampled analog input channels in ohm.

This method is deprecated (use [GDS_GNAUTILUS_GetImpedanceEx](#) instead).

[GDS_RESULT](#) GDS_GNAUTILUS_GetImpedance([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], double (*impedances)[[GDS_GNAUTILUS_CHANNELS_MAX](#)])

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to measure impedance for.
<i>deviceName</i>	A pointer to the name of the device to measure impedance for.
<i>impedances</i>	A pointer to an allocated array that receives the impedance values in ohm for each channel. An element at zero-based index <i>i</i> receives the impedance value in ohm of zero-based channel number <i>i</i> , or <i>std::numeric_limits<double>::quiet_NaN()</i> (see extended description of the method for details).

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>impedances</i> is NULL .
GDS_ERROR_UNKNOWN	Device does not support impedance measurement, or data acquisition is currently running, or impedance measurement is currently running, or an invalid reference signal is measured on electrode 'Cz' (maybe due to an unconnected electrode), or an error occurred during impedance measurement.

Remarks

Electrode 'Cz' plays a special role in impedance measurement and must always be connected. The method will fail if an invalid reference signal is measured on electrode 'Cz' (maybe due to an unconnected electrode), which does not allow calculation of impedance values of all other channels.

All returned impedance values are rounded to the nearest integral kilo-ohm value. All measured values are clipped within a range of 5,000 ohm to 500,000 ohm. However, if the impedance value cannot be measured exactly, the method returns a value of 4,000 ohm for any estimated impedance value below 5,000 ohm, and 501,000 for any estimated impedance value above 500,000 ohm. The method returns quiet *NaN* values (*std::numeric_limits<double>::quiet_NaN()*) for unavailable channels and for channels that did not yield a finite impedance value. The availability of a channel can be determined using [GDS GNAUTILUS GetAvailableChannels](#)

GDS_GNAUTILUS_GetImpedanceEx

Measures impedance of all available available synchronously sampled analog input channels in ohm.

[GDS_RESULT](#) GDS_GNAUTILUS_GetImpedanceEx([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], double (*impedances)[[GDS_GNAUTILUS_CHANNELS_MAX](#)])

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to measure impedance for.
<i>deviceName</i>	A pointer to the name of the device to measure impedance for.
<i>impedances</i>	A pointer to an allocated array that receives the impedance values in ohm for each channel. An element at zero-based index <i>i</i> receives the impedance value in ohm of zero-based channel number <i>i</i> , or <i>std::numeric_limits<double>::quiet_NaN()</i> (see extended description of the method for details).

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL , or <i>impedances</i> is NULL .
GDS_ERROR_UNKNOWN	Device does not support impedance measurement, or data acquisition is currently running, or impedance measurement is currently running, or an error occurred during impedance measurement.

Remarks

Electrode 'Cz' plays a special role in impedance measurement and must always be connected. The method will not fail if an invalid reference signal is measured on electrode 'Cz' (maybe due to a not connected electrode). Instead, appropriate error codes will be assigned to the other channels.

All returned impedance values are rounded to the nearest integral ohm value in range. All measured values are clipped within a range of 600 ohm to 500,000 ohm. Negative values represent different error codes. The method returns quiet *NaN* values (*std::numeric_limits<double>::quiet_NaN()*) for unavailable channels and for channels that did not yield a finite impedance value. The availability of a channel can be determined using [GDS GNAUTILUS GetAvailableChannels](#).

The following table lists the recommended classification of returned impedances for active gel electrodes:

Impedance value in ohm	Recommended classification for active gel electrodes
< 65,000	Good impedance.
65,000...200,000	Medium impedance.
> 200,000	Bad impedance.

The following table lists the description of returned error codes if the impedance value for a channel cannot be calculated:

Error code	Description
-1	The amplitude of the measured signal almost equals the original signal. The impedance is estimated to be very low.
-2	The impedance could not be calculated but is estimated to be very low.
-3	The amplitude of the measured signal is too small. The impedance is estimated to be very high.
-4	The offset of the measured signal is too big.
-7	The shape of the measured signal does not match the expected one.
-8	Too many invalid data frames were acquired during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.
-9	The amplitude of the measured signal at electrode 'Cz' is too small.
-10	The offset of the measured signal at electrode 'Cz' is too big.
-11	The shape of the measured signal at electrode 'Cz' does not match the expected one.
-12	Too many invalid data frames were acquired for electrode 'Cz' during impedance measurement. This indicates bad wireless connection between the g.Nautilus headset and base station.
<i>std::numeric_limits<double>::quiet_NaN()</i>	For unavailable channels and for channels that yielded non-finite impedance values (such as <i>NaN</i> or infinity).

GDS_GNAUTILUS_StartImpedanceMeasurement

Starts continuous impedance measurement asynchronously and calls the specified callback for each completed measurement cycle.

This method is deprecated (use [GDS_GNAUTILUS_StartImpedanceMeasurementEx](#) instead).

[GDS_RESULT](#) GDS_GNAUTILUS_StartImpedanceMeasurement([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GNAUTILUS_ImpedanceMeasurementCallback](#) impedanceMeasurementCallback, void *userData)

Parameters

<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to start impedance measurement for.
<i>deviceName</i>	A pointer to the name of the device to start impedance measurement for.
<i>impedanceMeasurementCallback</i>	The callback method to execute after each completed impedance measurement cycle.
<i>userData</i>	The user data which should be passed to the callback, or NULL if not used.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL .

Remarks

This method returns immediately. The *impedanceMeasurementCallback* function will be called after each completed impedance measurement cycle until the [GDS_GNAUTILUS_StopImpedanceMeasurement](#) method is called.

If an error occurs during impedance measurement, it is forwarded to the callback as parameter. Errors occur in the same situations as for [GDS_GNAUTILUS_GetImpedance](#).

If the *result* parameter in the [GDS_GNAUTILUS_ImpedanceMeasurementCallback](#) callback handler

indicates an error, and the *aborted* flag of the callback handler equals [*TRUE*](#), impedance measurement has been stopped automatically and can be started again with [*GDS GNAUTILUS StartImpedanceMeasurement*](#).

If the *result* parameter in the [*GDS GNAUTILUS ImpedanceMeasurementCallback*](#) callback handler indicates an error, but the *aborted* flag of the callback handler equals [*FALSE*](#), only the current measurement cycle is affected, but impedance measurement continues.

GDS_GNAUTILUS_StartImpedanceMeasurementEx

Starts continuous impedance measurement asynchronously and calls the specified callback for each completed measurement cycle.

[GDS_RESULT](#) GDS_GNAUTILUS_StartImpedanceMeasurementEx([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [GDS_GNAUTILUS_ImpedanceMeasurementCallback](#) impedanceMeasurementCallback, void *userData)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to start impedance measurement for.
<i>deviceName</i>	A pointer to the name of the device to start impedance measurement for.
<i>impedanceMeasurementCallback</i>	The callback method to execute after each completed impedance measurement cycle.
<i>userData</i>	The user data which should be passed to the callback, or NULL if not used.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL .

Remarks

This method returns immediately. The *impedanceMeasurementCallback* function will be called after each completed impedance measurement cycle until the [GDS_GNAUTILUS_StopImpedanceMeasurement](#) method is called.

If an error occurs during impedance measurement, it is forwarded to the callback as parameter. Errors occur in the same situations as for [GDS_GNAUTILUS_GetImpedanceEx](#).

If the *result* parameter in the [GDS_GNAUTILUS_ImpedanceMeasurementCallback](#) callback handler indicates an error, and the *aborted* flag of the callback handler equals [TRUE](#), impedance measurement has been stopped automatically and can be started again with

GDS GNAUTILUS StartImpedanceMeasurementEx.

If the *result* parameter in the [GDS GNAUTILUS ImpedanceMeasurementCallback](#) callback handler indicates an error, but the *aborted* flag of the callback handler equals [FALSE](#), only the current measurement cycle is affected, but impedance measurement continues.

GDS_GNAUTILUS_StopImpedanceMeasurement

Stops continuous, asynchronous impedance measurement started previously with [GDS_GNAUTILUS_StartImpedanceMeasurement](#).

[GDS_RESULT](#) GDS_GNAUTILUS_StopImpedanceMeasurement([GDS_HANDLE](#) connectionHandle, char (*deviceName)[[DEVICE_NAME_LENGTH_MAX](#)], [BOOL](#) waitUntilCompleted)

Parameters	
<i>connectionHandle</i>	The connection handle whose associated data acquisition session contains the device to stop impedance measurement for.
<i>deviceName</i>	A pointer to the name of the device to stop impedance measurement for.
<i>waitUntilCompleted</i>	TRUE to block until impedance measurement has been completed; FALSE to return immediately.

Returns

A [GDS_RESULT](#) structure containing information about the operation's success. It may represent one of the following errors:

GDS_ERROR_SUCCESS	The operation succeeded.
GDS_ERROR_INVALID_HANDLE	<i>connectionHandle</i> is invalid.
GDS_ERROR_HANDLE_NOT_ASSOCIATED	<i>connectionHandle</i> is not associated with any data acquisition session already opened with GDS_Connect .
GDS_ERROR_INVALID_DEVICE	<i>deviceName</i> refers to a device that is not part of the data acquisition session associated with <i>connectionHandle</i> .
GDS_INVALID_PARAMETER	<i>deviceName</i> is NULL .

g.NEEDACCESS
NETWORK ENABLED EASY DATA ACCESS